



Liepājas Valsts tehnikums

2D piedzīvojumu spēle “Gaismas Meklētājs”

Kvalifikācijas eksāmena praktiskās daļas dokumentācija

Darba autors:

Artjoms Sidorčenko, 4PT

Darba vadītājs:

Skolotājs, Kristaps Rāvalds

Eksāmena datums 2026. gada 16. un 17. jūnijs

Liepāja 2026

Saturs

Ievads.....	4
1. Uzdevuma formulējums.....	6
2. Programmatūras prasību specifikācija.....	8
2.1. Produkta perspektīva.....	8
2.2. Sistēmas funkcionālās prasības.....	8
2.3. Sistēmas nefunkcionālās prasības.....	22
2.4. Gala lietotāja raksturiezīmes.....	23
3. Izstrādes līdzekļu, rīku apraksts un izvēles pamatojums.....	24
3.1. Izvēlēto risinājuma līdzekļu un valodu apraksts.....	24
3.2. Iespējamo (<i>alternatīvo</i>) risinājuma līdzekļu un valodu apraksts.....	25
4. Sistēmas modelēšana un projektēšana.....	26
4.1. Sistēmas struktūras modelis.....	26
4.1.1. Sistēmas struktūra (komponenšu diagramma).....	26
4.1.2. Kļāšu diagramma.....	27
4.2. Funkcionālais un dinamiskais sistēmas modelis.....	28
4.2.1. Lietojumgadījumu diagramma (<i>Use Case</i>).....	28
4.2.2. Aktivitāšu diagramma (<i>Activity</i>).....	29
4.2.3. Stāvokļu diagramma (<i>State</i>).....	31
4.3. Datu struktūru apraksts.....	33
5. Lietotāju ceļvedis.....	35
6. Testēšanas dokumentācija.....	47
6.1. Izvēlētās testēšanas metodes, rīku apraksts un pamatojums.....	47
6.2. Testpiemēru kopa.....	48
6.3. Testēšanas žurnāls.....	53
Secinājumi.....	59
7. Lietoto terminu un saīsinājumu skaidrojumi.....	62
Literatūras un informācijas avotu saraksts.....	64

Ievads

Videospēles jau daudzus gadus ir nozīmīga izklaides un kultūras sastāvdaļa. Tās ietekmē gan jaunākās paaudzes, gan pieaugušos, kuri atceras, kāda bija sajūta spēlēt savas mīļākās spēles bērnībā. 2D piedzīvojumu spēles saglabā īpašu pievilcību — tās ir saprotamas, dinamiskas un piedāvā pakāpenisku progresu. Darba autors uzskata, ka šis žanrs ļauj veiksmīgi apvienot izaicinājumu un prieku vienlaikus.

Klasiskās spēles, piemēram, “Super Mario Bros” un “Sonic the Hedgehog”, radīja neaizmirstamu pieredzi ar dažādiem līmeņiem un unikālām spēles mehānikām. Tās aizrāva gan bērnus, gan jauniešus un pat pieaugušos, kuri atceras, kā bija spēlēt šādas spēles savā jaunībā. Savukārt spēles kā “Crash Bandicoot” piešķir dinamiku un piedzīvojuma sajūtu, lai gan to vizuālā struktūra atšķiras no tradicionālām 2D platform-mehānikām. Šīs spēles iedvesmoja arī šo projektu, kura mērķis ir panākt līdzīgu sajūtu spēlētājam.

Šī kvalifikācijas darba mērķis ir radīt 2D piedzīvojumu spēli ar vairākiem līmeņiem, kuros pakāpeniski tiek ieviestas jaunas mehānikas. Katrs līmenis būs atšķirīgs pēc dizaina un stilistikas, lai nodrošinātu daudzveidību un piedzīvojuma sajūtu. Darba autors uzskata, ka ir svarīgi, lai spēlētājs justos motivēts turpināt piedzīvojumu, pat ja kāds līmenis sākumā šķiet sarežģīts.

Spēlē būs arī veikala un skapja sadaļas, kurās spēlētājs varēs iegādāties un izvēlēties dažādus personāžus. Papildus tam spēlē būs paslēpti artefakti, kurus spēlētājs varēs atrast līmeņu laikā un apskatīt atsevišķā artefaktu kolekcijas sadaļā. Tas ļaus padarīt spēles pieredzi personiskāku un dos papildu motivāciju krāt zvaigznes un turpināt spēli. Šāda pieeja padara spēli interesantāku un veicina atkārtotu līmeņu spēlēšanu.

Personīgā motivācija šim darbam balstās uz bērnības pieredzi ar platformeru spēlēm. Tās sniedza gan prieku, gan izaicinājumus, attīstot stratēģisko domāšanu un reakciju. Projekts ir mēģinājums apvienot radošumu un tehniskās prasmes, radot spēli, kas varētu sniegt līdzīgu pieredzi nākamajām spēļu paaudzēm, vienlaikus atgādinot pieaugušajiem, kāda bija sajūta spēlēt šādas spēles viņu jaunībā.

Galvenais mērķis ir radīt spēli, kas ir interesanta, dinamiska un potenciāli gatava publicēšanai digitālajos izplatīšanas kanālos kā, piemēram, “Steam” vai “Epic Games”. Projekts ļauj apvienot radošo darbu ar praktisko pieredzi līmeņu izstrādē, spēles mehāniku veidošanā un testēšanā, kā arī demonstrē autora spēju plānot un īstenot pilnvērtīgu spēļu projektu.

Kvalifikācijas darbā ir detalizēti aprakstīts līmeņu dizains, spēles mehāniku izveide, vizuālā noformējuma izvēle, veikala un skapja funkcionalitātes realizācija, kā arī spēles progressa,

iestatījumu un rezultātu saglabāšanas risinājumi. Tas ļaus parādīt, kā no idejas tiek radīta gatava un interesanta spēle, kas spēj piesaistīt dažādu vecumu spēlētājus.

Kopumā šis projekts apvieno radošumu, spēļu dizainu un tehnisko izpildi, radot 2D piedzīvojumu spēli, kas piedāvā aizraujošu un daudzveidīgu pieredzi spēlētājiem, vienlaikus attīstot autora prasmes un zināšanas. Projekta attīstības gaitā visas spēles versijas un uzlabojumi tiek glabāti GitHub repozitorijā: <https://github.com/Deather2/GodotGame>. Tas ļauj sekot līdzi spēles progresam, salīdzināt dažādas versijas un viegli dalīties ar projektu.

1. Uzdevuma formulējums

Lai izstrādātu kvalifikācijas darba ietvaros paredzēto 2D platformera spēli, ir nepieciešams noteikt konkrētus uzdevumus, kas nodrošina pilnvērtīgu projekta izveidi — no idejas izstrādes līdz gatavam produktam. Uzdevuma formulējums palīdz sadalīt darbu vairākos posmos, plānot laiku un noteikt galvenos mērķus, kas jāīsteno spēles izstrādes procesā.

Galvenais šī darba uzdevums ir izstrādāt funkcionējošu 2D platformera spēli ar vairākiem līmeņiem, pakāpeniski pieaugošu sarežģītību un papildu spēles mehānikām. Spēlei jābūt tehniski stabilai, vizuāli pievilcīgai un interesantai plašākai auditorijai. Lai sasniegtu šo mērķi, ir izvirzīti vairāki konkrēti uzdevumi:

1. Idejas un koncepcijas izstrāde.

Šajā posmā tiek precizēta spēles ideja, tās stils, mērķauditorija un galvenās spēles mehānikas. Autors nosaka spēles vidi, sižetu un varoņu raksturojumu. Tiek veikta līdzīgu spēļu analīze, lai saprastu, kādi elementi padara šāda tipa spēles aizraujošas un kā tos iespējams pilnveidot.

2. Līmeņu un mehāniku plānošana.

Autors izveido līmeņu struktūru, nosakot to skaitu, sarežģītību un vizuālo stilu. Katram līmenim ir paredzēts savs izaicinājums vai spēles elements, piemēram, kustīgās platformas, kontrolpunkti, interaktīvi objekti un citi līmeņa elementi. Šāda pieeja pakāpeniski paplašina spēlētāja pieredzi un padara līmeņu iziešanu daudzveidīgāku.

3. Spēles vides un objektu izveide.

Autors izstrādā spēles vizuālos elementus, piemēram, fonus, platformas, personāžus un citus spēles objektus. Īpaša uzmanība tiek pievērsta vizuālajai saskaņotībai un estētikai, lai spēlei būtu patīkams un saprotams dizains.

4. Spēles loģikas un mehāniku programmēšana.

Šajā posmā autors īsteno varoņa kustību, ienaidnieku uzvedību, sadursmju noteikšanu, kustīgo platformu darbību, kontrolpunktu sistēmu, interaktīvo objektu loģiku un citu spēles mehānismu darbību. Tiek nodrošināts, lai spēle darbotos vienmērīgi, bez būtiskām kļūdām un ar atbilstošu reakciju uz spēlētāja darbībām.

5. Veikala un pielāgojamu elementu ieviešana.

Viens no nozīmīgākajiem uzdevumiem ir veikala un skapja funkcionalitātes izstrāde. Tajās spēlētājs var iegādāties jaunus personāžus un pēc tam izvēlēties tos skapja sadaļā. Tas prasa gan tehnisko risinājumu izstrādi, gan vizuālā noformējuma plānošanu, lai sistēma būtu saprotama un ērta lietotājam.

6. Testēšana un kļūdu labošana.

Kad spēles galvenā funkcionalitāte ir pabeigta, autors veic testēšanu, lai pārliecinātos par stabilitāti un lietošanas ērtumu. Tiek pārbaudītas spēles mehānikas, varoņa kustība, sadursmes, kontrolpunkti, kustīgās platformas, ienaidnieki, interaktīvie objekti, pārejas starp līmeņiem, iestatījumi, veikala un skapja darbība. Ja tiek atklātas kļūdas, tās tiek novērstas pirms spēles uzskatīšanas par pabeigtu.

7. Dokumentācijas sagatavošana.

Šo uzdevumu autors veic paralēli visiem pārējiem, sagatavojot detalizētu dokumentāciju par spēles izstrādes procesu, tehniskajiem risinājumiem un iegūtajiem rezultātiem. Tas ļauj skaidri parādīt, kā tiek plānota un realizēta katra projekta daļa.

Veicot visus šos uzdevumus, tiek sasniegts kvalifikācijas darba mērķis — radīt pilnvērtīgu un interesantu 2D platformera spēli, kas apvieno radošumu un tehnisko izpildījumu, demonstrējot autora spējas gan dizaina, gan programmēšanas jomā.

2. Programmatūras prasību specifikācija

Šajā nodaļā tiek aprakstītas galvenās prasības, kas nosaka spēles funkcionalitāti. Tiek sniegts pārskats par programmatūras perspektīvu, sistēmas funkcionālajām un nefunkcionālajām prasībām. Šīs prasības palīdzēs nodrošināt, ka spēle tiek izstrādāta atbilstoši mērķiem, ir lietotājam draudzīga un tehniski stabila. Nodaļā turpmāk detalizēti tiks izklāstīts, kādas funkcijas, īpašības un lietotāja iespējas spēlē ir paredzētas, kā arī kā tiek nodrošināta sistēmas kvalitāte un efektivitāte.

2.1. Produkta perspektīva

Spēles perspektīva balstās uz tās paplašināmību un turpmāko attīstību. Projekts ir izstrādāts tā, lai tam varētu pievienot jaunus līmeņus, personāžus, ienaidniekus, interaktīvus objektus un papildu spēles mehānikas, būtiski nepārveidojot esošo sistēmas struktūru. Spēle ir potenciāli gatava publicēšanai digitālajos izplatīšanas kanālos, piemēram, “Steam” vai “Epic Games”, kas ļautu sasniegt plašu auditoriju. Turklāt projekta struktūra nodrošina iespēju attīstīt turpmākus paplašinājumus, papildinot gan vizuālo dizainu, gan spēles funkcionalitāti. Perspektīva projektam ir kombinēt radošu pieeju ar tehnisko attīstību, veidojot platformu ilgtermiņa spēles uzlabošanai un jaunu satura pievienošanai.

2.2. Sistēmas funkcionālās prasības

FP.1. Galvenās izvēlnes attēlošana

Mērķis:

Parādīt galveno izvēlni, lai spēlētājs varētu piekļūt spēles sadaļām.

Ievaddati:

Spēle tiek palaista.

Apstrāde:

- 1) Spēle inicializē sākuma vidi (kameru, apgaismojumu, UI elementus).
- 2) Sistēma ielādē galvenās izvēlnes dizaina elementus (fonu, logotipu, pogas).
- 3) Sistēma pārbauda, vai pastāv saglabātie dati.
- 4) Aktivizē fona mūziku, kas piesaistīta galvenajam skatam.
- 5) Sagatavo pārejas efektus uz citām sadaļām.

Izvaddati:

- 1) Uz ekrāna tiek attēlota galvenā izvēlne ar visiem funkcionāliem elementiem.
- 2) Lietotājs var ar peli vai tastatūru atlasīt jebkuru sadaļu (“Sākt spēli”, “Veikals” u.c.).
- 3) Fona mūzika tiek atskaņota, un izvēlne reaģē uz lietotāja darbībām (hover efekti, klikšķi).

FP.2. Spēles sākšana

Mērķis:

Nodrošināt spēles sākšanu no galvenās izvēlnes.

Ievaddati:

Lietotājs nospiež pogu “Sākt spēli”.

Apstrāde:

Sistēma pāriet uz līmeņu izvēles skatu.

Izvaddati:

Uz ekrāna parādās līmeņu saraksts.

FP.3. Līmeņu saraksta attēlošana

Mērķis:

Parādīt visus pieejamos un bloķētos līmeņus, lai lietotājs varētu izvēlēties, kuru spēlēt.

Ievaddati:

Lietotājs nonāk līmeņu izvēles skatā pēc tam, kad nospiesta poga “Sākt spēli”.

Apstrāde:

- 1) Sistēma nolasa lokālos progresu datus (bloķētie / atbloķētie līmeņi, zvaigznes).
- 2) Tiek ģenerēts līmeņu saraksts un katram līmenim piešķirts statuss (“pieejams”, “bloķēts”).
- 3) Pieejamajiem līmeņiem tiek pievienota iespēja uzklikšķināt un sākt spēli, bloķētajiem – redzams slēdzenes simbols.

Izvaddati:

- 1) Uz ekrāna redzami visi līmeņi ar statusiem un zvaigžņu skaitu.
- 2) Bloķētajiem līmeņiem redzama pelēka ikona ar slēdzeni.
- 3) Pieejamie līmeņi ir izgaismoti un reaģē uz peles klikšķi.

FP.4. Līmeņu atbloķēšanas mehānisms

Mērķis:

Atbloķēt nākamo līmeni pēc iepriekšējā pabeigšanas, pamatojoties uz spēlētāja rezultātiem.

Ievaddati:

Spēlētājs pabeidz līmeni vismaz ar vienu zvaigzni.

Apstrāde:

- 1) Sistēma pārbauda, kurš līmenis ir pabeigts un kāds rezultāts sasniegts.
- 2) Ja līmenis pabeigts sekmīgi, sistēma atjaunina progresu failā.
- 3) Nākamais līmenis tiek atzīmēts kā “atbloķēts”.
- 4) Ja visi līmeņi ir pabeigti, sistēma parāda paziņojumu par spēles pabeigšanu.

Izvaddati:

Nākamais līmenis kļūst pieejams līmeņu sarakstā.

FP.5. Zvaigžņu vērtēšanas sistēma

Mērķis:

Novērtēt spēlētāja sniegumu ar zvaigznēm (0–3), pamatojoties uz sasniegto rezultātu.

Ievaddati:

- 1) Līmenis pabeigts.
- 2) Saglabāts līmeņa rezultāts (piemēram, laiks, dzīvību skaits u.c.).

Apstrāde:

- 1) Sistēma aprēķina rezultātu, ņemot vērā līmeņa izpildes laiku un spēlētāja kļūdu skaitu.
- 2) Tiek noteikts zvaigžņu skaits no 1 līdz 3.
- 3) Rezultāts tiek saglabāts progresā datos.
- 4) Ja jaunais rezultāts ir labāks par iepriekšējo, sistēma to aizstāj.
- 5) Rezultātu logā tiek parādīts iegūtais zvaigžņu skaits, līmeņa izpildes laiks un cita svarīga statistika.

Izvaddati:

- 1) Ekrānā tiek parādīts sasniegtais zvaigžņu skaits.
- 2) Uz līmeņa ikonas līmeņu sarakstā parādās jaunais vērtējums.

FP.6. Līmeņa sākšana

Mērķis:

Nodrošināt konkrēta līmeņa palaišanu.

Ievaddati:

Lietotājs nospiež uz pieejamā līmeņa.

Apstrāde:

Sistēma ielādē izvēlēto līmeni.

Izvaddati:

Spēles aina tiek palaista.

FP.7. Pauzes izvēlne

Mērķis:

Ļaut spēlētājam apturēt spēli un piekļūt papildu opcijām, nezaudējot progresu.

Ievaddati:

Lietotājs nospiež “Esc” spēles laikā.

Apstrāde:

- 1) Sistēma aptur spēles fiziku, laiku un varoņa kustību.
- 2) Tiek apturēta fona mūzika vai samazināts tās skaļums.

3) Uz ekrāna tiek attēlots modālais logs ar izvēlēm “Turpināt”, “Izspēlēt pa jaunu”, “Iestatījumi”, “Galvenā izvēlne”.

4) Ja lietotājs nospiež pogu “Izspēlēt pa jaunu”, sistēma aizver pauzes izvēlni un atkārtoti ielādē pašreizējo līmeni.

5) Ja lietotājs izvēlas “Iestatījumi”, tiek ielādēts iestatījumu logs.

6) Ja lietotājs nospiež “Turpināt”, spēle atsākas no tās pašas vietas.

7) Ja lietotājs nospiež “Galvenā izvēlne”, sistēma aizver pauzes izvēlni un atver galveno izvēlni.

Izvaddati:

Parādās modālais logs ar pogām “Turpināt”, “Izspēlēt pa jaunu”, “Iestatījumi”, “Galvenā izvēlne”.

FP.8. Līmeņa rezultātu logs

Mērķis:

Parādīt spēlētājam detalizētu rezultātu pēc līmeņa iziešanas, iekļaujot zvaigznes un pogas nākamajām darbībām.

Ievaddati:

1) Līmenis pabeigts.

2) Saglabāts rezultāts.

Apstrāde:

1) Sistēma aprēķina zvaigžņu skaitu pēc noteiktajiem kritērijiem.

2) Sistēma saglabā rezultātu lokālajā progresā failā.

3) Rezultātu logā tiek attēlots iegūtais zvaigžņu skaits, līmeņa izpildes laiks un spēlētāja kļūdu vai nāvju skaits.

4) Tiek parādītas darbību pogas, kas ļauj pāriet uz nākamo līmeni, atkārtot pašreizējo līmeni vai atgriezties galvenajā izvēlnē.

Izvaddati:

1) Ekrānā tiek parādīts rezultātu logs ar zvaigznēm un līmeņa statistiku.

2) Pieejamas pogas “Nākamais līmenis”, “Izspēlēt pa jaunu”, “Galvenā izvēlne”, kas reaģē uz klikšķiem.

FP.9. Progresā saglabāšana

Mērķis:

Automātiski saglabāt spēlētāja progresu pēc līmeņa pabeigšanas.

Ievaddati:

Līmenis pabeigts.

Apstrāde:

Sistēma saglabā zvaigznes, līmeņa statusu, labākos rezultātus, iegādātos personāžus un atrastos artefaktus lokālajā progresā failā.

Izvaddati:

Progresu, iegādātos personāžus un atrastos artefaktus var redzēt nākamajā spēles palaišanas reizē.

FP.10. Progresā atiestatīšana

Mērķis:

Atļaut lietotājam atiestatīt progresu.

Ievaddati:

Lietotājs nospiež pogu "Atiestatīt progresu".

Apstrāde:

Sistēma dzēš saglabātos datus.

Izvaddati:

Pirmais līmenis paliek pieejams, pārējie līmeņi kļūst bloķēti, zvaigznes pazūd, iegādātie personāži vairs nav pieejami, un atrastie artefakti tiek atiestatīti.

FP.11. Iestatījumu logs

Mērķis:

Ļaut lietotājam atvērt iestatījumu logu un mainīt spēles iestatījumus.

Ievaddati:

Lietotājs nospiež "Iestatījumi".

Apstrāde:

- 1) Sistēma nolasa pašreizējos iestatījumus no saglabāšanas faila.
- 2) Atver iestatījumu logu ar vairākām sadaļām: video, vadības un audio iestatījumiem.

Izvaddati:

Uz ekrāna tiek attēlots iestatījumu logs, un lietotājs var mainīt iestatījumus reāllaikā.

FP.12. Skaņas skaļuma kontrole

Mērķis:

Nodrošināt skaņas līmeņa pielāgošanu un saglabāšanu.

Ievaddati:

Lietotājs pārvieto skaļuma slīdni.

Apstrāde:

- 1) Sistēma reģistrē jauno skaļuma vērtību izvēlētajai audio kategorijai.
- 2) Tiek atjaunināts attiecīgais audio skaļuma līmenis spēlē, piemēram, izvēlnes mūzikai, skaņas efektiem un citām audio kategorijām.
- 3) Sistēma saglabā jauno vērtību iestatījumu datus.

Izvaddati:

Skaņa mainās reāllaikā atbilstoši jaunajai vērtībai.

FP.13. Video iestatījumu kontrole

Mērķis:

Nodrošināt video iestatījumu maiņu un to saglabāšanu.

Ievaddati:

Lietotājs atver iestatījumu logu un maina video iestatījumus.

Apstrāde:

- 1) Sistēma piemēro izvēlētās video iestatījumu vērtības.
- 2) Mainītās vērtības tiek saglabātas lokālajā iestatījumu failā.
- 3) Pēc atkārtotas spēles palaišanas sistēma ielādē iepriekš saglabātos video iestatījumus.

Izvaddati:

Video iestatījumi tiek piemēroti un saglabāti turpmākai lietošanai.

FP.14. Veikala attēlošana

Mērķis:

Parādīt spēles veikalu ar pieejamajiem personāžiem un to cenām.

Ievaddati:

Lietotājs nospiež pogu “Veikals”.

Apstrāde:

- 1) Sistēma ielādē veikalā pieejamos personāžus un to cenas zvaigznēs.
- 2) Sistēma pārbauda spēlētāja pieejamo zvaigžņu skaitu.
- 3) Jau iegādātie personāži tiek atzīmēti atšķirīgi no vēl neiegādātajiem.

Izvaddati:

Ekrānā redzams veikals ar personāžiem, to cenām un darbību pogām.

FP.15. Pirkuma veikšana veikalā

Mērķis:

Nodrošināt iespēju iegādāties jaunu personāžu par zvaigznēm.

Ievaddati:

Lietotājs nospiež pogu “Pirkt” pie izvēlētā personāža.

Apstrāde:

- 1) Sistēma pārbauda, vai spēlētājam ir pietiekams zvaigžņu skaits.
- 2) Ja zvaigžņu pietiek, sistēma atņem nepieciešamo daudzumu un atzīmē personāžu kā iegādātu.
- 3) Ja zvaigžņu nepietiek, pirkums netiek veikts.

Izvaddati:

Iegādātais personāžs tiek atzīmēts kā nopirkts un kļūst pieejams skapī.

FP.16. Skapja attēlošana

Mērķis:

Parādīt spēlētāja iegādātos personāžus.

Ievaddati:

Lietotājs nospiež pogu “Skapis”.

Apstrāde:

- 1) Sistēma ielādē spēlētāja iegādāto personāžu sarakstu.
- 2) Tiek attēlots, kurš personāžs pašlaik ir izvēlēts.

Izvaddati:

Uz ekrāna redzams skapja saturs ar pieejamajiem personāžiem.

FP.17. Personāža izvēle

Mērķis:

Nodrošināt iespēju izvēlēties vienu no iegādātajiem personāžiem lietošanai spēlē.

Ievaddati:

Lietotājs skapī izvēlas iegādāto personāžu.

Apstrāde:

- 1) Sistēma saglabā izvēlēto personāžu kā aktīvo.
- 2) Nākamajā spēles palaišanas reizē tiek izmantots izvēlētais personāžs.

Izvaddati:

Spēlē tiek attēlots izvēlētais personāžs.

FP.18. Atgriešanās uz galveno izvēlni

Mērķis:

Nodrošināt ātru pāreju uz galveno izvēlni.

Ievaddati:

Lietotājs nospiež “Atpakaļ” vai “Galvenā izvēlne”.

Apstrāde:

- 1) Sistēma sagatavo pāreju uz sākuma ekrānu (ielādē fonu, pogas, UI elementus).
- 2) Fona mūzika tiek pārslēgta uz galvenās izvēlnes celiņu.

Izvaddati:

Uz ekrāna tiek attēlota galvenā izvēlne ar visiem funkcionāliem elementiem.

FP.19. Spēles aizvēršana

Mērķis:

Droši aizvērt spēli.

Ievaddati:

Lietotājs nospiež “Iziet”.

Apstrāde:

- 1) Sistēma saglabā pēdējo progresu.
- 2) Pārtrauc spēles procesus un audio atskaņošanu.

Izvaddati:

Spēle tiek aizvērta bez datu zuduma.

FP.20. Skaņas efekti

Mērķis:

Pievienot audio atgriezenisko saiti pogām un darbībām.

Ievaddati:

Lietotājs nospiež pogu vai veic darbību.

Apstrāde:

- 1) Sistēma atskaņo attiecīgo skaņas efektu.
- 2) Skaļums atbilst iestatījumiem.

Izvaddati:

Lietotājs dzird skaņu atbilstoši darbībai.

FP.21. Fona mūzika

Mērķis:

Nodrošināt nepārtrauktu fona mūziku dažādos skatos.

Ievaddati:

Skats tiek ielādēts.

Apstrāde:

- 1) Sistēma atskaņo skatam specifisku mūzikas celiņu.
- 2) Mūzika turpinās starp skatiem, ja tas ir loģiski.

Izvaddati:

Fona mūzika skan atbilstoši skaļuma iestatījumiem.

FP.22. Nākamā līmeņa ielāde

Mērķis:

Piedāvāt ātru pāreju uz nākamo līmeni pēc rezultātu loga.

Ievaddati:

Lietotājs nospiež “Nākamais līmenis” rezultātu loga.

Apstrāde:

Sistēma ielādē nākamo līmeni.

Izvaddati:

Tiek palaists nākamais līmenis bez atgriešanās līmeņu sarakstā.

FP.23. Līmeņa atkārtota ielāde no rezultātu loga

Mērķis:

Ļaut spēlētājam ātri atkārtoti palaist pašreizējo līmeni no rezultātu loga.

Ievaddati:

Lietotājs rezultātu logā nospiež pogu “Izspēlēt pa jaunu”.

Apstrāde:

- 1) Sistēma aizver rezultātu logu.
- 2) Sistēma atkārtoti ielādē pašreizējo līmeni.

Izvaddati:

Pašreizējais līmenis tiek palaists no jauna bez atgriešanās citos skatos.

FP.24. Automātiska progresu ielāde

Mērķis:

Ielādēt saglabāto progresu spēles startā.

Ievaddati:

Spēle tiek palaista.

Apstrāde:

- 1) Sistēma pārbauda lokālos saglabāšanas failus.
- 2) Ielādē līmeņu statusus, zvaigznes, iegādātos personāžus, izvēlēto personāžu un

atrasto artefaktu statusu.

Izvaddati:

Līmeņu skats, iestatījumi, skapis un artefaktu kolekcijas sadaļa attēlo aktuālo progresu.

FP.25. Galvenā varoņa kontrole

Mērķis:

Nodrošināt lietotājam iespēju vadīt galveno varoni, izmantojot iestatījumos norādītos vadības taustiņus.

Ievaddati:

Lietotājs nospiež kustības, lēciena, pietupšanās, mijiedarbības vai uzbrukuma vadības taustiņus.

Apstrāde:

- 1) Spēle nepārtraukti apstrādā lietotāja ievades darbības.
- 2) Atbilstoši nospiestajām darbībām varonis pārvietojas pa kreisi vai pa labi, lec, pietupjas, mijiedarbojas ar tuvumā esošu objektu vai boss cīņas laikā izpilda uzbrukumu.
- 3) Varoņa kustība tiek veikta, ņemot vērā fizikas motoru, sadursmes un gravitāciju.

Izvaddati:

Galvenais varonis izpilda atbilstošās kustības saskaņā ar lietotāja ievadi.

1. tabula

Galvenā varoņa vadības taustiņi pēc noklusējuma

Taustiņš	Darbība
W vai Space	Lēciens
A	Varoņa kustība pa kreisi
D	Varoņa kustība pa labi
S vai Ctrl	Pietupšanās
Esc	Pauze
E	Mijiedarbība ar objektiem
Peles kreisā poga vai Enter	Uzbrukums boss cīņās laikā

FP.26. Vadības taustiņu maiņa

Mērķis:

Ļaut lietotājam mainīt spēles vadības taustiņus un saglabāt tos turpmākai lietošanai.

Ievaddati:

Lietotājs atver iestatījumu loga vadības sadaļu un izvēlas darbību, kurai jāmaina taustiņš.

Apstrāde:

- 1) Sistēma gaida jaunu lietotāja ievadi izvēlētajai darbībai.
- 2) Pēc taustiņa vai peles pogas nospiešanas sistēma piešķir jauno ievadi konkrētajai darbībai.
- 3) Ja izvēlētajā ievade jau ir piešķirta citai darbībai, sistēma nepieļauj dublēšanos un attēlo atbilstošu paziņojumu.
- 4) Mainītie vadības taustiņi tiek saglabāti lokālajā iestatījumu failā.
- 5) Ja spēles laikā tiek rādītas vadības norādes, tās tiek atjauninātas atbilstoši jaunajām taustiņu vērtībām.
- 6) Pēc atkārtotas spēles palaišanas sistēma ielādē iepriekš saglabātos vadības taustiņus.

Izvaddati:

Lietotājs redz atjaunotos vadības taustiņus iestatījumu logā, un spēlē tiek izmantotas jaunās vadības vērtības.

FP.27. Vadības iestatījumu atiestatīšana uz noklusējumu

Mērķis:

Ļaut lietotājam atiestatīt mainītos vadības taustiņus uz noklusējuma vērtībām.

Ievaddati:

Lietotājs atver iestatījumu loga vadības sadaļu un nospiež pogu vadības atiestatīšanai uz noklusējumu.

Apstrāde:

1) Sistēma atjauno visām vadības darbībām sākotnēji paredzētos taustiņus un peles pogas.

2) Atjaunotās vadības vērtības tiek saglabātas lokālajā iestatījumu failā.

3) Ja iestatījumu logs ir atvērts, lietotājs uzreiz redz atjaunotās noklusējuma vērtības.

Izvaddati:

Vadības sadaļā tiek attēloti noklusējuma taustiņi, un spēlē turpmāk tiek izmantotas noklusējuma vadības vērtības.

FP.28. Interaktīvo objektu izmantošana

Mērķis:

Nodrošināt iespēju spēlētājam mijiedarboties ar īpašiem objektiem līmeņa laikā.

Ievaddati:

Lietotājs atrodas pie interaktīva objekta un nospiež mijiedarbības taustiņu.

Apstrāde:

1) Sistēma pārbauda, vai spēlētājs atrodas interaktīvā objekta darbības zonā.

2) Ja spēlētājs atrodas objekta tuvumā, tiek parādīta mijiedarbības norāde.

3) Pēc mijiedarbības taustiņa nospiešanas sistēma izpilda objektam paredzēto darbību.

4) Atkarībā no objekta veida sistēma var piešķirt spēlētājam pastiprinājumu, aktivizēt animāciju vai sākt līmeņa pabeigšanas plūsmu.

Izvaddati:

Interaktīvais objekts izpilda tam paredzēto darbību, un spēlētājs redz atbilstošu rezultātu spēles laikā.

FP.29. Kontrolpunktu sistēma

Mērķis:

Nodrošināt spēlētāja atdzimšanu pēdējā aktivizētajā kontrolpunktā pēc varoņa nāves.

Ievaddati:

Spēlētājs līmeņa laikā sasniedz kontrolpunktu.

Apstrāde:

1) Sistēma pārbauda, vai spēlētājs ir saskāries ar kontrolpunktu.

2) Ja kontrolpunkts vēl nav aktivizēts, sistēma to aktivizē.

3) Sistēma saglabā aktivizēto kontrolpunktu kā jauno atdzimšanas vietu konkrētajā līmenī.

4) Ja spēlētājs nomirst, sistēma atdzīvina varoni pēdējā aktivizētajā kontrolpunktā.

Izvaddati:

Pēc nāves varonis atdzimst pēdējā aktivizētajā kontrolpunktā, nevis līmeņa sākumā.

FP.30. Īslaicīga pastiprinājuma sistēma

Mērķis:

Nodrošināt iespēju spēlētājam iegūt īslaicīgu spējas pastiprinājumu līmeņa laikā.

Ievaddati:

Spēlētājs atrodas pie pastiprinājuma objekta un nospiež mijiedarbības taustiņu.

Apstrāde:

- 1) Sistēma pārbauda, vai spēlētājs atrodas pastiprinājuma objekta darbības zonā.
- 2) Pēc mijiedarbības taustiņa nospiešanas sistēma aktivizē pastiprinājumu.
- 3) Pastiprinājums uz noteiktu laiku maina spēlētāja īpašības, piemēram, palielina lēciena augstumu.
- 4) Pastiprinājuma darbības laikā tiek attēlots vizuāls efekts.
- 5) Pēc noteiktā laika beigām vai pēc spēlētāja nāves pastiprinājums tiek atcelts.

Izvaddati:

Spēlētājs īslaicīgi iegūst pastiprinātu spēju, un pēc pastiprinājuma beigām varoņa īpašības atgriežas sākotnējā stāvoklī.

FP.31. Ienaidnieku sistēma

Mērķis:

Nodrošināt līmeņos ienaidniekus, kas rada spēlētājam papildu šķēršļus un palielina spēles izaicinājumu.

Ievaddati:

Spēlētājs līmeņa laikā tuvojas ienaidniekam vai saskaras ar to.

Apstrāde:

- 1) Sistēma nodrošina ienaidnieka pārvietošanos pa noteiktu maršrutu.
- 2) Ja ienaidnieks sasniedz šķērslī vai noteiktu robežu, tas maina kustības virzienu.
- 3) Sistēma pārbauda sadursmi starp spēlētāju un ienaidnieku.
- 4) Ja spēlētājs saskaras ar ienaidnieka bīstamo zonu, tiek aktivizēta varoņa nāves loģika.

Izvaddati:

Ienaidnieks pārvietojas līmenī un rada draudu spēlētājam. Saskares gadījumā varonis zaudē mēģinājumu un tiek atdzīvināts atbilstoši līmeņa loģikai.

FP.32. Kustīgo platformu sistēma

Mērķis:

Nodrošināt līmeņos kustīgas platformas, kas paplašina spēles mehānikas un rada papildu izaicinājumu spēlētājam.

Ievaddati:

Spēlētājs līmeņa laikā uzlec uz kustīgas platformas vai izmanto to pārvietošanās procesā.

Apstrāde:

- 1) Sistēma pārvieto platformu pa iepriekš noteiktu virzienu vai maršrutu.
- 2) Platforma maina kustības virzienu, sasniedzot noteiktu robežu vai galapunktu.
- 3) Ja spēlētājs atrodas uz platformas, sistēma nodrošina, ka varonis pārvietojas kopā ar platformu.
- 4) Platformas kustība tiek apstrādāta kopā ar spēlētāja fiziku un sadursmēm.

Izvaddati:

Kustīgā platforma darbojas līmenī, un spēlētājs var to izmantot, lai pārvarētu šķēršļus vai sasniegtu citas līmeņa daļas.

FP.33. Līmeņa pabeigšana ar kristālu

Mērķis:

Nodrošināt līmeņa pabeigšanu, izmantojot īpašu interaktīvu objektu - kristālu.

Ievaddati:

Spēlētājs atrodas pie kristāla un nospiež mijiedarbības taustiņu.

Apstrāde:

- 1) Sistēma pārbauda, vai spēlētājs atrodas kristāla darbības zonā.
- 2) Ja spēlētājs nospiež mijiedarbības taustiņu, tiek aktivizēta kristāla animācija.
- 3) Pēc kristāla aktivizēšanas sistēma izpilda līmenim paredzēto noslēguma plūsmu.
- 4) Sistēma aptur līmeņa taimeri un sagatavo rezultātu logu.
- 5) Tiek aprēķināts un saglabāts spēlētāja rezultāts.

Izvaddati:

Līmenis tiek pabeigts, spēlētāja rezultāts tiek saglabāts, un uz ekrāna tiek parādīts rezultātu logs.

FP.34. Boss cīņas sistēma

Mērķis:

Nodrošināt finālajā līmenī boss cīņu, kurā spēlētājs var izmantot uzbrukuma darbību, lai uzvarētu boss pretinieku un turpinātu līmeņa noslēguma plūsmu.

Ievaddati:

- 1) Spēlētājs sasniedz boss zonas sākumu.
- 2) Boss cīņa tiek aktivizēta pēc ievada notikuma.
- 3) Lietotājs boss cīņas laikā nospiež uzbrukuma taustiņu.

Apstrāde:

- 1) Sistēma bloķē boss arēnu un novieto kameru boss cīņas zonā.
- 2) Tiek parādīta boss veselības josla.
- 3) Boss izpilda uzbrukumus atkarībā no spēlētāja atrašanās vietas.

- 4) Spēlētājs var uzbrukt boss pretiniekam tikai boss cīņas laikā.
- 5) Ja spēlētājs nomirst boss cīņas laikā, sistēma atiestata boss stāvokli un atdzīvina spēlētāju boss cīņas kontrolpunktā.
- 6) Kad boss veselība sasniedz nulli, tiek palaista boss nāves animācija un turpināta finālā līmeņa noslēguma plūsma.

Izvaddati:

- 1) Boss cīņa norisinās aktīvā spēles līmenī.
- 2) Spēlētājs redz boss veselības joslu un var samazināt boss veselību ar uzbrukumiem.
- 3) Pēc boss uzvarēšanas spēlētājs var turpināt ceļu līdz finālajam kristālam.

FP.35. Spēles fināla un titru attēlošana

Mērķis:

Nodrošināt spēles noslēguma plūsmu pēc finālā kristāla aktivizēšanas.

Ievaddati:

- 1) Finālais boss ir uzvarēts.
- 2) Spēlētājs sasniedz finālo kristālu.
- 3) Lietotājs nospiež mijiedarbības taustiņu pie kristāla.

Apstrāde:

- 1) Sistēma aktivizē finālā kristāla notikumu.
- 2) Tiek atjaunota spēles pasaules gaisma.
- 3) Tiek apturēta spēlētāja vadība un sākas fināla pāreja.
- 4) Sistēma parāda titrus ar projekta autoru un izmantoto resursu autoriem.
- 5) Pēc titru parādīšanas tiek attēlots rezultātu logs bez pogas "Nākamais līmenis".

finālā līmeņa noslēguma plūsma.

Izvaddati:

- 1) Spēlētājs redz spēles noslēguma titrus.
- 2) Pēc titriem tiek parādīts finālā līmeņa rezultātu logs.
- 3) Lietotājs var atkārtot finālo līmeni vai atgriezties izvēlnē.

FP.36. Slēpto artefaktu kolekcija

Mērķis:

Nodrošināt iespēju spēlētājam atrast slēptus artefaktus līmeņu laikā un apskatīt tos atsevišķā kolekcijas sadaļā.

Ievaddati:

- 1) Spēlētājs atrodas pie slēptā artefakta līmeņa laikā.
- 2) Lietotājs nospiež mijiedarbības taustiņu pie artefakta.
- 3) Lietotājs galvenajā izvēlnē atver artefaktu sadaļu.

Apstrāde:

- 1) Sistēma pārbauda, vai spēlētājs atrodas slēptā artefakta darbības zonā.
- 2) Ja spēlētājs nospiež mijiedarbības taustiņu, artefakts tiek atzīmēts kā atrasts.
- 3) Artefakta statuss tiek saglabāts lokālajā progresā failā.
- 4) Pēc artefakta savākšanas tiek parādīts paziņojums par atrasto artefaktu.
- 5) Artefaktu kolekcijas sadaļā sistēma attēlo visus artefaktus un to statusu: "Atrasts"

vai "Nav atrasts".

- 6) Ja artefakts jau ir atrasts, tas līmenī atkārtoti netiek parādīts.

Izvaddati:

- 1) Atrastais artefakts tiek saglabāts spēlētāja progresā.
- 2) Uz ekrāna tiek parādīts paziņojums par artefakta atrašanu.
- 3) Artefaktu kolekcijas sadaļā tiek attēlots aktuālais atrasto artefaktu skaits un katra artefakta statuss.

2.3. Sistēmas nefunkcionālās prasības

NP.1. Veiktspēja

Spēlei jāstrādā ar vismaz 60 kadriem sekundē.

NP.2. Saderība

Spēlei jābūt saderīgai vismaz ar Windows operētājsistēmu.

NP.3. Lietojamība

Lietotāja interfeisam jābūt saprotamam un ērti izmantojamam bez apmācības.

NP.4. Datu drošība

Lietotāja iestatījumi un progress jāglabā droši, bez bojājumiem.

NP.5. Atjaunošana

Spēlei jābūt viegli papildināmai ar jauniem līmeņiem un funkcijām.

NP.6. Estētika

Grafikai un animācijām jābūt vienotā stilā un vizuāli pievilcīgām.

NP.7. Koda kvalitāte

Projekta kodam jābūt strukturētam un viegli uzturamam.

NP.8. Atmiņas patēriņš

Spēles atmiņas patēriņam jābūt ne vairāk kā 1 GB.

NP.9. Ielādes indikators

Ielādes laikā jābūt redzamam progresam vai animācijai.

NP.10. Starta logs

Spēlei jāuzsākas ar galveno izvēlni bez kavēšanās vai papildu ielādes ekrāniem.

2.4. Gala lietotāja raksturiezīmes

Spēles “Gaismas Meklētājs” gala lietotājs ir cilvēks, kurš interesējas par 2D piedzīvojumu un platformas spēlēm. Spēle ir piemērota lietotājiem, kuriem patīk līmeņu iziešana, šķēršļu pārvarēšana, precīza varoņa vadība un pakāpeniski pieaugošs izaicinājums. Galvenā mērķauditorija ir pusaudži un jaunieši, taču spēli var spēlēt ikviens lietotājs, kurš vēlas izbaudīt klasisku 2D platformera tipa spēli ar piedzīvojuma elementiem. Spēle paredzēta izklaidei, tāpēc tai jābūt saprotamai, vizuāli pievilcīgai un pietiekami ērtai lietošanai bez īpašas apmācības.

3. Izstrādes līdzekļu, rīku apraksts un izvēles pamatojums

Šajā nodaļā tiek aprakstīti visi rīki un tehnoloģijas, kas tika izmantoti spēles “Gaismas Meklētājs” izstrādes procesā, kā arī sniegts pamatojums to izvēlei. Autors analizē izmantoto programmēšanas valodu, izstrādes vidi un citas nepieciešamās programmas, kas palīdzēja realizēt projekta mērķus. Tiek izskaidrots, kāpēc konkrētie risinājumi tika izvēlēti, ņemot vērā projekta prasības, autora zināšanu līmeni un tehniskās iespējas.

Tāpat šajā nodaļā tiek apskatīti arī alternatīvie risinājumi — citi rīki un valodas, kas potenciāli varētu tikt izmantoti līdzīga tipa spēles izstrādei. Tiek norādīti to plusi un mīnusi, salīdzinot ar izvēlēto risinājumu. Šī analīze palīdz pamatot, ka izvēlēta pieeja ir visefektīvākā konkrētajam projektam un atbilst gan funkcionālajām, gan nefunkcionālajām prasībām.

3.1. Izvēlēto risinājuma līdzekļu un valodu apraksts

Lai nodrošinātu kvalitatīvu un efektīvu spēles izstrādes procesu, darbā tiek izmantoti vairāki rīki un tehnoloģijas, kas savstarpēji papildina viens otru un ļauj sasniegt vēlamu rezultātu.

1. Godot Engine

Galvenā izstrādes vide ir Godot Engine. Tā ir moderna, atvērta koda spēļu izstrādes vide, kas piedāvā plašas iespējas 2D projektu veidošanai. Godot ir ērts interfeiss, viegli pārskatāma struktūra un ļoti labs 2D atbalsts, kas ir būtiski šī projekta ietvaros. Šis dzinējs ļauj izveidot spēles mehānikas, animācijas un vadību vienotā vidē, neizmantojot ārējos papildinājumus.

2. GDScript

Kā programmēšanas valoda tiek izmantots GDScript, kas ir speciāli izstrādāta Godot dzinējam. Tā ir līdzīga Python sintaksei, kas padara to viegli apgūstamu un ļauj koncentrēties uz loģiku, nevis tehniskām detaļām. GDScript nodrošina labu veikspēju un ātru izstrādes tempu, kas ir būtiski kvalifikācijas darba ietvaros.

3. GitHub

Lai nodrošinātu versiju kontroli un failu drošu glabāšanu, tiek izmantots GitHub. Tas ļauj saglabāt projekta izmaiņu vēsturi, atgriezties pie iepriekšējām versijām un nodrošina ērtu sadarbību un rezerves kopiju izveidi.

4. Aseprite

Aseprite tiek izmantots spēles vizuālo elementu izveidei. Tas ir specializēts rīks pikseļu grafikas un animāciju veidošanai, kas ir īpaši piemērots retro stila 2D spēlēm. Ar tā palīdzību iespējams izstrādāt spēles varoņu, objektu un vides elementu dizainu.

Apvienojot visus šos rīkus — Godot Engine, GDScript, Visual Studio Code, GitHub un Aseprite — tiek nodrošināta pilnvērtīga un efektīva izstrādes vide, kas ļaus izveidot kvalitatīvu 2D platformera spēli no idejas līdz gatavam produktam.

3.2. Iespējamo (*alternatīvo*) risinājuma līdzekļu un valodu apraksts

Izstrādes procesa sākumā tika apsvērti vairāki citi spēļu dzinēji un programmēšanas risinājumi, kuri teorētiski varētu nodrošināt līdzīgu rezultātu kā Godot Engine, tomēr katram no tiem bija arī savi trūkumi, kas padarīja tos mazāk piemērotus šī projekta vajadzībām.

Unity (C#) ir viens no populārākajiem spēļu dzinējiem pasaulē, taču tā izmantošana prasa vairāk laika, jo vide ir salīdzinoši sarežģīta un resursu prasīga. Unity ir lielisks risinājums 3D spēlēm, taču 2D projektiem tas bieži vien ir pārāk apjomīgs, un daudzas funkcijas paliek neizmantotas. Turklāt bezmaksas versijā ir noteikti ierobežojumi, kas var radīt problēmas turpmākai projekta attīstībai.

Unreal Engine (C++) piedāvā ļoti augstu veiktspēju un grafisko kvalitāti, tomēr tas ir daudz komplicētāks rīks, kas vairāk piemērots lieliem un tehniski sarežģītiem projektiem. C++ valoda ir sarežģītāka, tās apgūšanai nepieciešams vairāk laika, un tas nebūtu efektīvi šī darba ietvaros, kur mērķis ir izstrādāt 2D platformera tipa spēli.

Python + Pygame ir vienkāršs un piemērots mazākiem projektiem vai mācību nolūkiem, tomēr tam ir ierobežotas grafiskās un veiktspējas iespējas. Pygame nav optimizēts lielākām spēlēm vai animācijām, un tas prasa manuālu darbu ar daudzām pamatfunkcijām, kas citos dzinējos jau ir iebūvētas.

JavaScript + Phaser tika apsvērts kā iespēja tīmekļa spēles izstrādei, tomēr šajā gadījumā spēle tiek paredzēta darbam uz Windows platformas, tāpēc šāds risinājums nebūtu piemērots. Turklāt Phaser vairāk orientēts uz vieglām, pārlūkprogrammā darbināmām spēlēm, nevis uz pilnvērtīgiem lokāliem projektiem ar plašākām iespējām.

Apkopojot, visi minētie risinājumi ir funkcionāli un spēcīgi savās jomās, taču tie nebija optimāli izvēlētajam mērķim — 2D platformera izveidei Windows vidē, kur galvenais uzsvars ir uz izstrādes vienkāršību, ātru testēšanu un radošu pieeju, ko vislabāk nodrošina Godot Engine.

4. Sistēmas modelēšana un projektēšana

Šajā nodaļā ir aprakstīta spēles sistēmas modelēšana un projektēšana. Tās mērķis ir parādīt, kā tika plānota spēles uzbūve, galvenās funkcijas un to savstarpējā saistība. Šajā nodaļā tiek apskatīta sistēmas struktūra, galvenie skati, lietotāja darbības un svarīgākie sistēmas elementi, kas nodrošina spēles darbību.

Sistēmas modelēšana un projektēšana ir svarīga, jo tā palīdz saprast, kā spēle ir izveidota un kā darbojas tās atsevišķās daļas. Šajā nodaļā tiks parādīti izmantotie modeļi un aprakstīti galvenie projektēšanas risinājumi, kas tika izmantoti spēles izstrādes laikā.

4.1. Sistēmas struktūras modelis

Šajā apakšnodaļā tiek aprakstīts spēles “Gaismas Meklētājs” sistēmas struktūras modelis. Tās mērķis ir parādīt, no kādām galvenajām daļām sastāv izstrādātā sistēma un kā šīs daļas ir savstarpēji saistītas. Sistēmas struktūras modelis palīdz labāk saprast spēles uzbūvi, tās galvenos komponentus un to savstarpējo sadarbību.

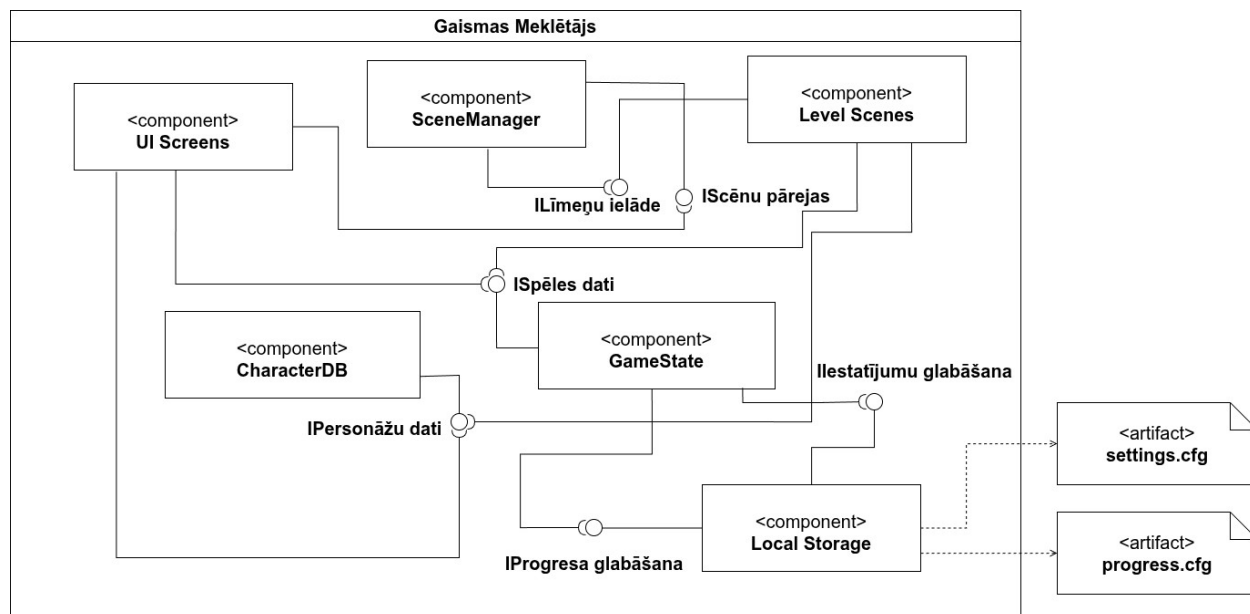
Šajā apakšnodaļā tiks aplūkota sistēmas kopējā struktūra, atsevišķi sistēmas elementi, kā arī to loma spēles darbībā. Tiks parādīts, kā ir organizēti galvenie skati, vadības elementi, spēles loģika un datu glabāšanas risinājumi. Šāds apraksts ļauj pārskatāmi attēlot sistēmas tehnisko uzbūvi un atvieglo turpmāku spēles analīzi un pilnveidošanu.

4.1.1. Sistēmas struktūra (komponenšu diagramma)

Šajā apakšnodaļā ir attēlota spēles “Gaismas Meklētājs” komponenšu diagramma (Skat. 1. attēlu.). Šī diagramma tika izvēlēta tāpēc, ka tā vislabāk parāda sistēmas galvenās sastāvdaļas, to savstarpējo saistību un izmantotās saskarnes. Tā kā spēle ir lokāla 2D platformera spēle, kas darbojas vienā datorā, izvietojuma diagramma šajā gadījumā nebūtu tik informatīva. Komponenšu diagramma ļauj pārskatāmi parādīt sistēmas uzbūvi un galveno komponentu sadarbību.

Diagrammā ir attēloti galvenie sistēmas komponenti: UI Screens, SceneManager, Level Scenes, GameState, CharacterDB un Local Storage. UI Screens komponents apvieno spēles galvenos lietotāja saskarnes skatus: galveno izvēlni, līmeņu izvēlni, veikalu, skapi, artefaktu kolekcijas sadaļu, iestatījumus, pauzes izvēlni un rezultātu logu. SceneManager nodrošina pārejas starp dažādām ainām, savukārt Level Scenes komponents ietver spēles līmeņus, galveno varoni, līmeņu objektus, ienaidniekus, kontrolpunktus, kustīgās platformas, līmeņa pabeigšanas loģiku un citus spēles līmeņos izmantotos elementus. GameState atbild par spēles progresu, iestatījumu un izvēlēto datu pārvaldību, bet CharacterDB satur personāžu datus, kas tiek izmantoti spēles gaitā. Local Storage nodrošina datu saglabāšanu un ielādi.

Diagrammā ir parādīti arī sistēmas izmantotie interfeisi, kas raksturo komponentu savstarpējo sadarbību. Tie parāda, kurš komponents nodrošina noteiktu funkcionalitāti un kurš komponents to izmanto. Atsevišķi ārpus sistēmas robežas ir attēloti artefakti settings.cfg un progress.cfg, kuros tiek saglabāti lietotāja iestatījumi un spēles progress. Šāda komponentu diagramma ļauj skaidri saprast sistēmas struktūru un tās galveno daļu sadarbību.

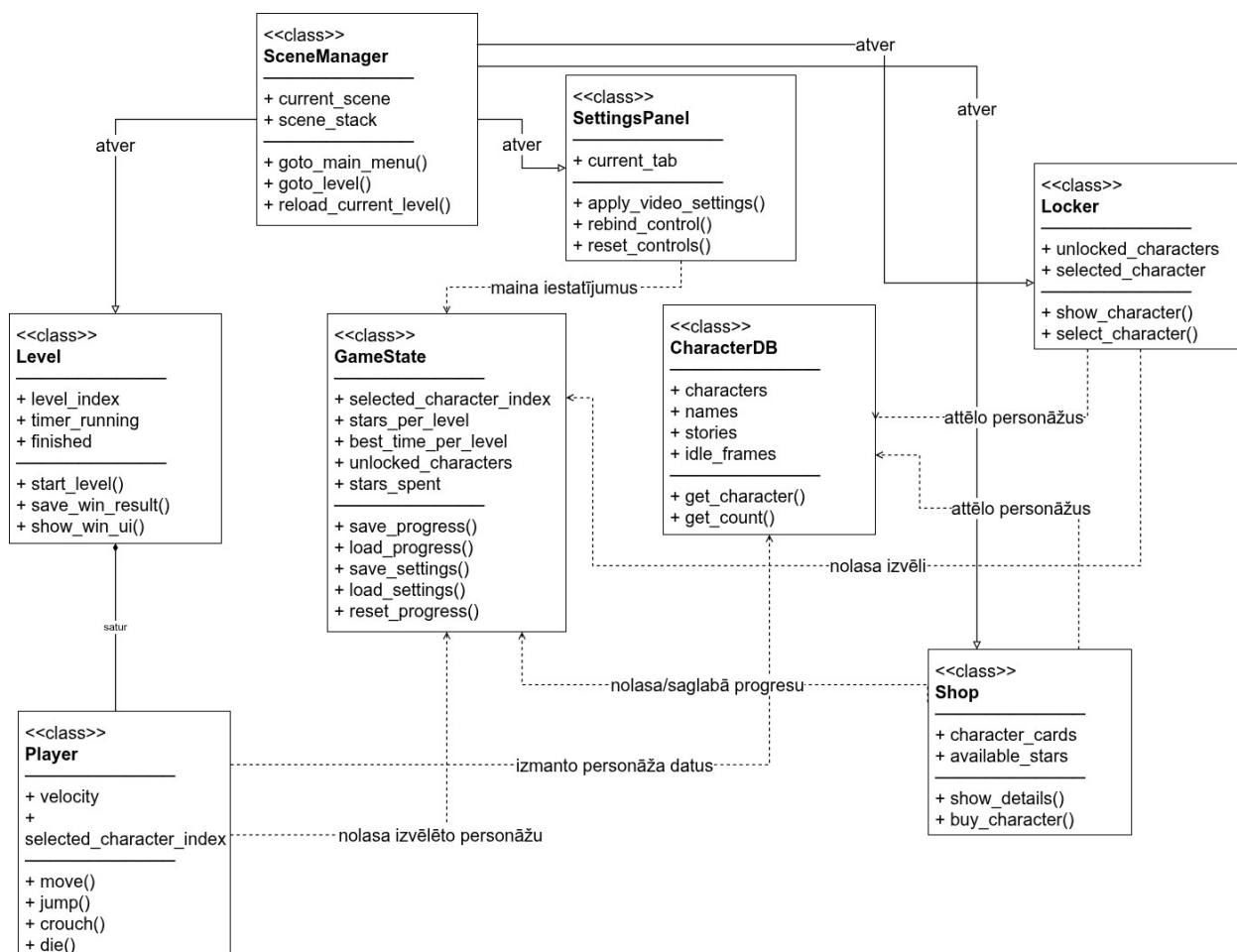


1. attēls. Komponentu diagramma

4.1.2. Klašu diagramma

Šajā apakšnodaļā ir attēlota spēles “Gaismas Meklētājs” klašu diagramma (Skat. 2. attēlu.). Tā tika izvēlēta tāpēc, ka šī diagramma vislabāk parāda sistēmas galvenās programmatūras klases, to atribūtus, metodes un savstarpējās saites. Šajā projektā svarīgāk ir parādīt spēles loģikas objektus un to sadarbību, jo spēle izmanto lokālu datu saglabāšanu failos settings.cfg un progress.cfg, nevis relāciju datubāzi.

Diagrammā ir attēlotas svarīgākās sistēmas klases: SceneManager, Level, Player, GameState, CharacterDB, SettingsPanel, Shop un Locker. SceneManager nodrošina pārejas starp spēles skatiem un līmeņiem. Level satur galveno varoni un līmeņa darbības loģiku, kā arī izmanto vairākus atkārtoti lietojamus līmeņa objektus, piemēram, kontrolpunktus, interaktīvus objektus un citus līmeņa elementus. Player izmanto GameState un CharacterDB datus, lai ielādētu izvēlēto personāžu un apstrādātu tā kustības. GameState pārvalda spēles progresu, zvaigznes, labākos laikus, atbloķētos personāžus un iestatījumus. SettingsPanel ļauj mainīt video, audio un vadības iestatījumus, savukārt Shop un Locker izmanto GameState un CharacterDB, lai attēlotu personāžus, veiktu iegādi un nodrošinātu personāža izvēli. Šāda klašu diagramma palīdz pārskatāmi saprast sistēmas iekšējo uzbūvi un galveno klašu sadarbību spēles darbības nodrošināšanā.



2. attēls. Klašu diagramma

4.2. Funkcionālais un dinamiskais sistēmas modelis

Šajā apakšnodaļā ir aprakstīts spēles “Gaismas Meklētājs” funkcionālais un dinamiskais sistēmas modelis. Tā mērķis ir parādīt, kā lietotājs mijiedarbojas ar sistēmu, kādas galvenās darbības tajā ir iespējamas un kā tās tiek izpildītas spēles darbības laikā. Šis modelis palīdz labāk saprast sistēmas uzvedību, lietotāja iespējas un darbību secību dažādās situācijās.

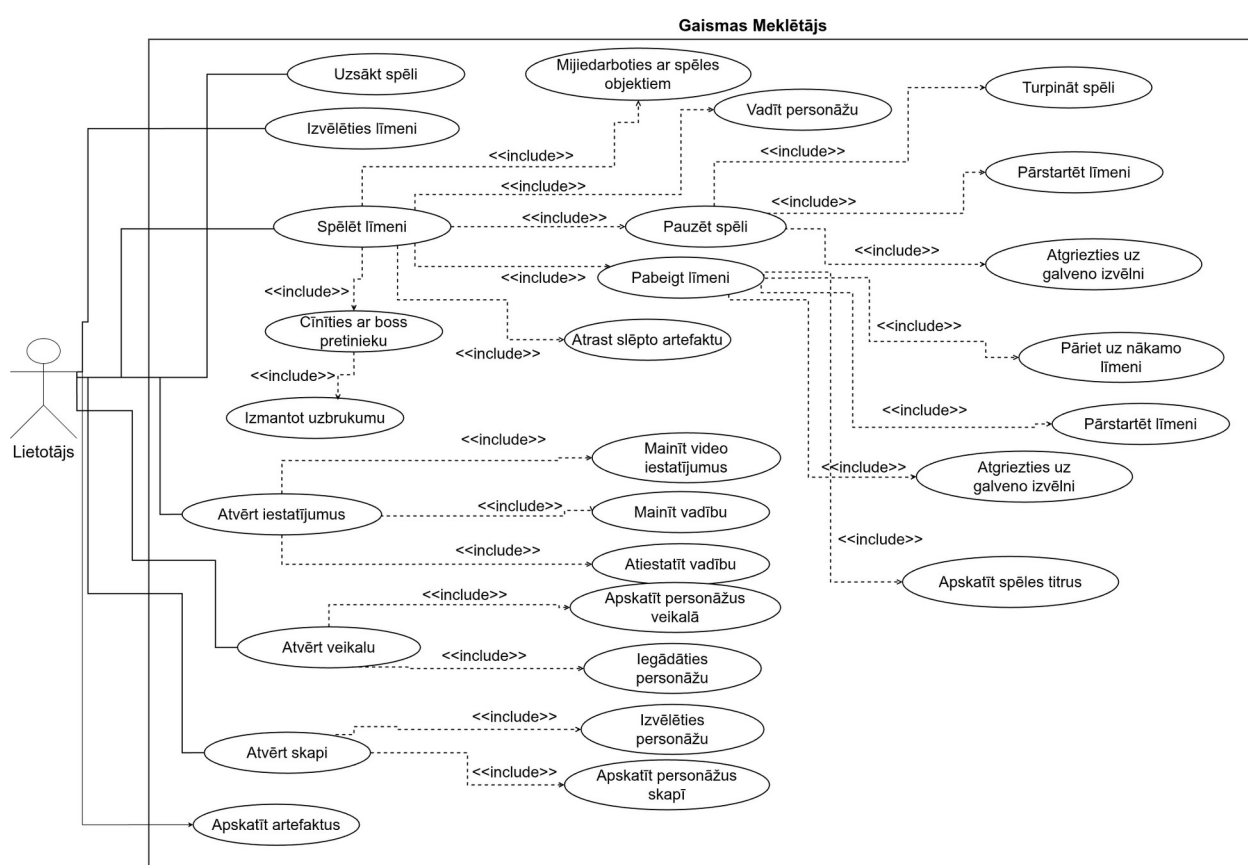
Apakšnodaļā tiks aplūkoti galvenie lietojumgadījumi, aktivitātes un sistēmas stāvokļi. Tajā būs parādīts, kā lietotājs uzsāk spēli, pārvietojas starp izvēlnēm, maina iestatījumus, izvēlas līmeni, kontrolē varoni un pabeidz spēles līmeņus. Šāds apraksts ļauj pārskatāmi attēlot sistēmas darbību gan no lietotāja, gan no pašas sistēmas skatpunkta, kā arī atvieglo spēles loģikas analīzi un turpmāku pilnveidošanu.

4.2.1. Lietojumgadījumu diagramma (Use Case)

Šajā apakšnodaļā ir attēlota spēles “Gaismas Meklētājs” lietojumgadījumu diagramma (Skat. 3. attēlu.). Tā parāda galvenās darbības, kuras lietotājs var veikt sistēmā, kā arī šo darbību saistību ar spēles funkcionalitāti. Lietojumgadījumu diagramma palīdz pārskatāmi saprast sistēmu no lietotāja skatpunkta un parāda, kā lietotājs mijiedarbojas ar spēles galvenajām sadaļām.

Diagrammā ir attēlots viens galvenais aktors — lietotājs. Lietotājs var sākt spēli, izvēlēties līmeni, spēlēt līmeni, atvērt iestatījumus, apmeklēt veikalu, atvērt skapi un apskatīt artefaktu kolekciju. Šīs darbības raksturo galvenās spēles sadaļas un lietotāja iespējas ārpus pašas līmeņa spēlēšanas.

Papildus galvenajām darbībām diagrammā ir parādīti arī detalizētāki lietojumgadījumi, kas saistīti ar spēles procesu. Tie ietver galvenā varoņa vadīšanu, mijiedarbību ar līmeņa objektiem, pauzes izvēlnes izmantošanu, līmeņa pabeigšanu, rezultātu apskati, personāžu iegādi un izvēli, kā arī finālā līmeņa notikumus. Šie lietojumgadījumi atbilst dokumentā aprakstītajām funkcionālajām prasībām un palīdz parādīt, kā spēles funkcijas ir pieejamas lietotājam.

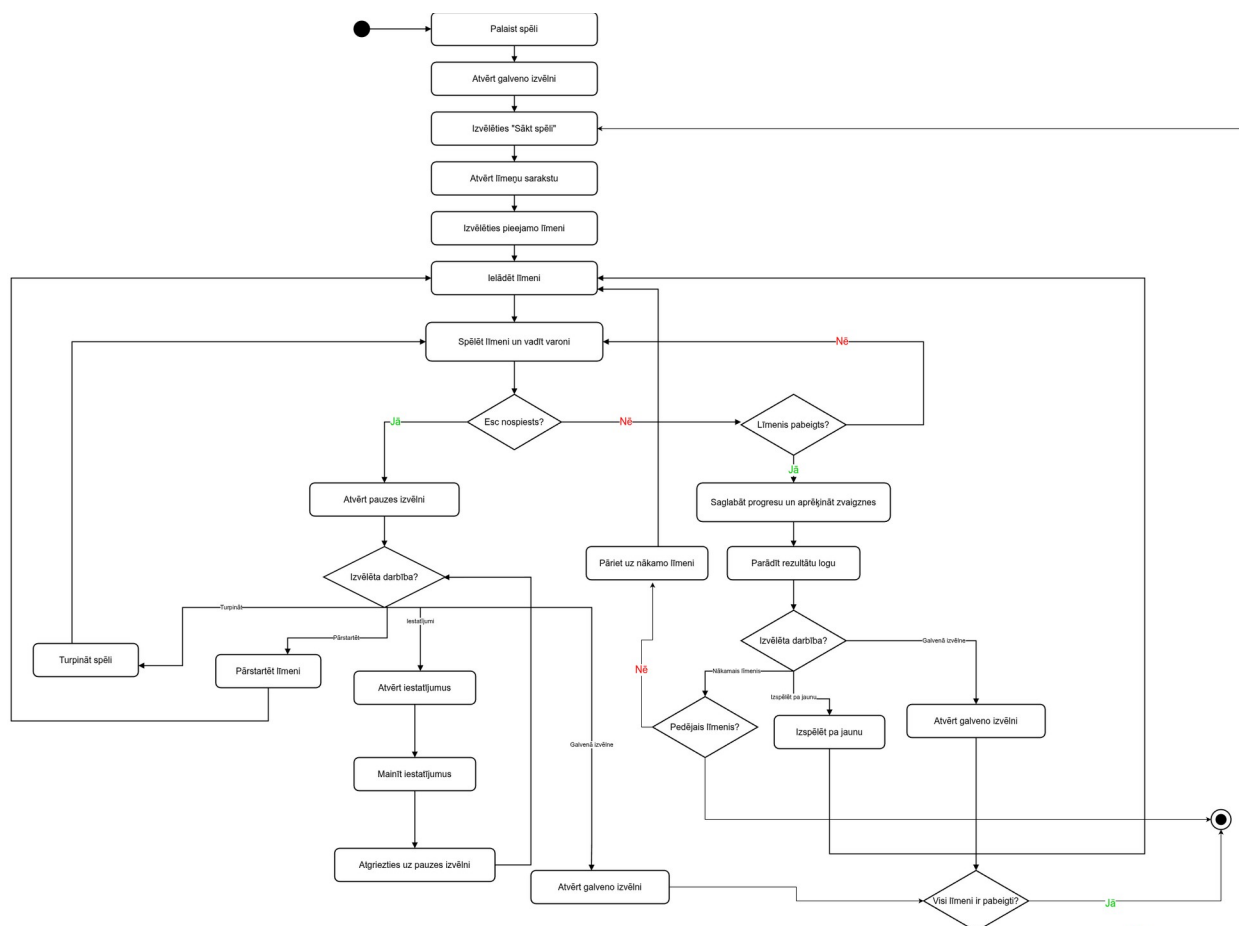


3. attēls. Lietojumgadījumu diagramma

4.2.2. Aktivitāšu diagramma (Activity)

Šajā apakšnodaļā ir attēlotas spēles “Gaismas Meklētājs” aktivitāšu diagrammas (Skat. 4. un 5. attēlu.). Tās parāda galveno darbību secību sistēmas darbības laikā un palīdz saprast, kā tiek apstrādātas svarīgākās spēles plūsmas. Tā kā projektā ir gan kopējā spēles plūsma starp izvēlnēm un līmeņiem, gan atsevišķa galvenā varoņa vadības loģika līmeņa laikā, aktivitāšu modelis ir sadalīts divās diagrammās. Šāds sadalījums padara attēlojumu pārskatāmāku un ļauj vieglāk uztvert gan sistēmas kopējo darbību secību, gan spēlētāja darbību apstrādi.

Pirmajā diagrammā (Skat. 4. attēlu.) ir attēlota galvenā spēles plūsma. Diagrammā redzams, ka lietotājs palaiž spēli, atver galveno izvēlni, izvēlas sākt spēli, atver līmeņu sarakstu un izvēlas pieejamo līmeni. Pēc tam sistēma ielādē līmeni, un sākas spēles process. Spēles laikā lietotājs var atvērt pauzes izvēlni, turpināt spēli, pārstartēt līmeni, atvērt iestatījumus vai atgriezties uz galveno izvēlni. Ja līmenis tiek pabeigts, sistēma saglabā progresu, aprēķina zvaigznes un parāda rezultātu logu, no kura iespējams pāriet uz nākamo līmeni, izspēlēt līmeni pa jaunu vai atgriezties uz galveno izvēlni.



4. attēls. SceneManager aktivitāšu diagramma

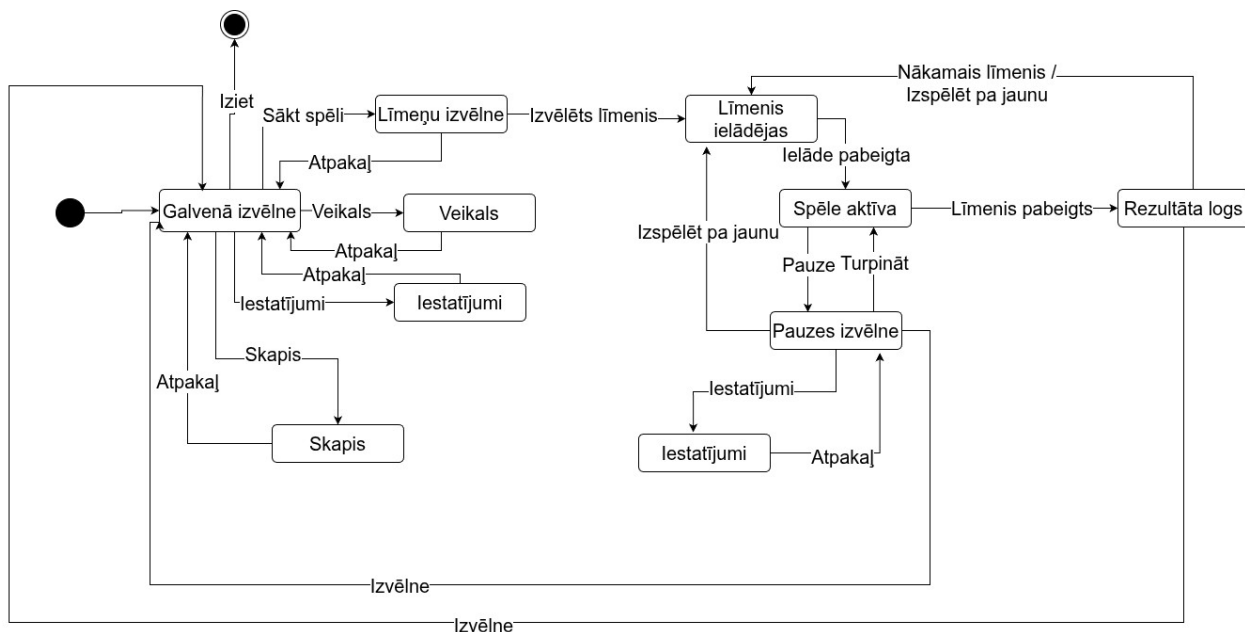
Otrajā diagrammā (Skat. 5. attēlu.) ir attēlota galvenā varoņa darbību secība līmeņa laikā. Diagrammā redzams, ka pēc līmeņa un personāža ielādes sistēma nepārtraukti apstrādā lietotāja ievadi. Tiek pārbaudīts, vai varonis ir sasniedzis nāves zonu, un šādā gadījumā tiek izpildīta nāves animācija un varonis atdzimst sākuma punktā vai pēdējā aktivizētajā kontrolpunktā. Ja nāves zona nav sasniegta, sistēma pārbauda, vai ir nospiests lēciena taustiņš, pietupiena taustiņš, kustības taustiņi, mijiedarbības taustiņš vai boss cīņas laikā uzbrukuma taustiņš. Atbilstoši ievadei varonis lec, pietupjas, pārvietojas pietupienā, pārvietojas pa kreisi vai pa labi, mijiedarbojas ar tuvumā esošu objektu, izpilda uzbrukumu boss cīņas laikā vai paliek uz vietas. Pēc darbības izvēles sistēma pielieto gravitāciju un sadursmes, kā arī atjaunina animāciju. Ja varonis saskaras ar kontrolpunktu, sistēma atjaunina atdzimšanas vietu. Ja tiek aktivizēts interaktīvs līmeņa objekts, sistēma izpilda

5. attēls. Player aktivitāšu diagramma

4.2.3. Stāvokļu diagramma (*State*)

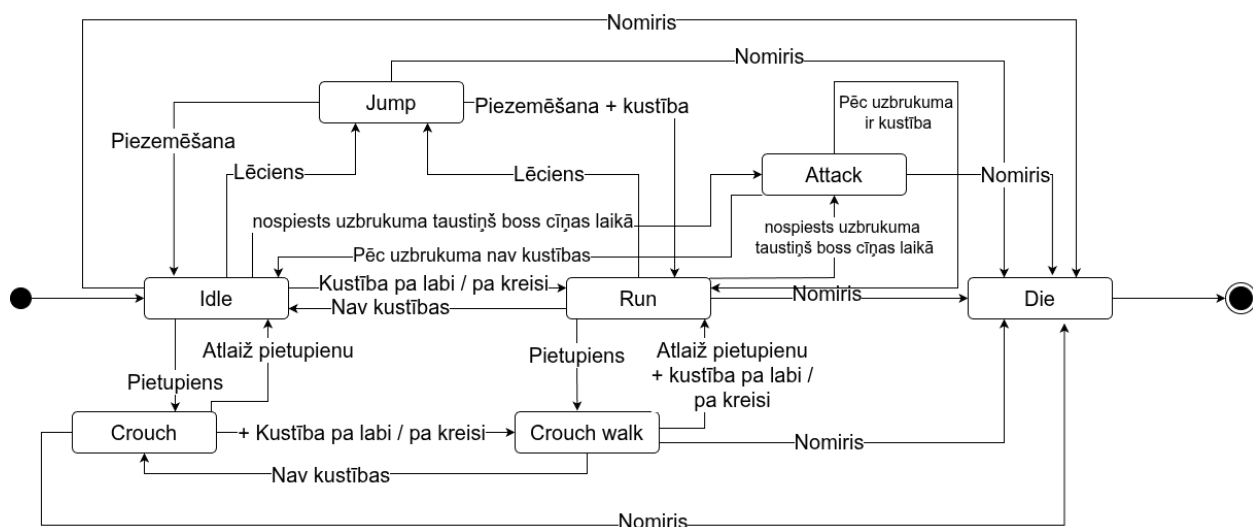
Šajā apakšnodaļā ir attēlotas spēles “Gaismas Meklētājs” stāvokļu diagrammas (Skat. 6. un 7. attēlu.). Tās parāda, kā sistēma un atsevišķi tās objekti maina savus stāvokļus atkarībā no lietotāja darbībām un notikumiem spēles laikā. Šādas diagrammas palīdz pārskatāmi saprast spēles dinamisko uzvedību, pārejas starp galvenajām ainām un galvenā varoņa stāvokļu maiņu līmeņa laikā.

Pirmajā diagrammā (Skat. 6. attēlu.) ir attēloti spēles galvenie stāvokļi un pārejas starp tiem, kuras pārvalda SceneManager. Diagrammā redzams, ka no galvenās izvēlnes lietotājs var pāriet uz līmeņu izvēlni, veikalu, skapi vai iestatījumiem. Pēc līmeņa izvēles sistēma pāriet uz līmeņa ielādes stāvokli, pēc tam uz aktīvas spēles stāvokli. Spēles laikā iespējams atvērt pauzes izvēlni, turpināt spēli, atvērt iestatījumus, izspēlēt līmeni pa jaunu vai pēc līmeņa pabeigšanas pāriet uz rezultātu logu. No rezultātu loga lietotājs var pāriet uz nākamo līmeni vai izspēlēt līmeni vēlreiz. Šī diagramma palīdz saprast kopējo spēles plūsmu un pārejas starp galvenajiem sistēmas stāvokļiem.



6. attēls. SceneManager stāvokļu diagramma

Otrajā diagrammā (Skat. 7. attēlu.) ir attēloti galvenā varoņa stāvokļi līmeņa laikā. Tā parāda, kā spēlētāja objekts pāriet starp stāvokļiem Idle, Run, Jump, Fall, Crouch, Crouch walk, Attack un Dead. Pārejas notiek atkarībā no spēlētāja ievades un situācijas spēlē, piemēram, kustības, lēciena, pietupiena, uzbrukuma boss cīņas laikā un citiem spēles notikumiem. Stāvoklis Attack tiek izmantots tikai boss cīņas laikā, kad spēlētājs nospiež uzbrukuma taustiņu un tiek atskaņota uzbrukuma animācija. Pēc uzbrukuma animācijas beigām varonis atgriežas Idle vai Run stāvoklī atkarībā no tā, vai spēlētājs turpina kustību. Pēc nāves varonis atdzimst sākuma punktā vai pēdējā aktivizētajā kontrolpunktā. Šajā diagrammā stāvokļu nosaukumi ir atstāti angļu valodā, jo tādi paši nosaukumi tiek izmantoti arī pašā projektā. Diagramma palīdz pārskatāmi saprast galvenā varoņa uzvedību un tā stāvokļu maiņu spēles gaitā.



7. attēls. Player stāvokļu diagramma

4.3. Datu struktūru apraksts

Šajā apakšnodaļā tiek aprakstītas spēlē “Gaismas Meklētājs” izmantotās datu struktūras un datu tipi, kas nodrošina spēles progresu, iestatījumu, personāžu un ainu pārvaldību. Tā kā spēle ir lokāls viena lietotāja projekts un neizmanto relāciju datubāzi, datu glabāšana un apstrāde tiek organizēta ar iekšējām programmēšanas struktūrām, resursiem un lokāliem saglabāšanas failiem. Šāda pieeja ir piemērota konkrētajam projektam, jo tā nodrošina vienkāršu datu pārvaldību, pietiekamu veiktspēju un ērtu sistēmas paplašināšanu nākotnē.

Viena no svarīgākajām datu pārvaldības daļām projektā ir **GameState**. Šajā struktūrā tiek glabāti galvenie spēles progresu un iestatījumu dati, piemēram, izvēlētais personāžs, līmeņu rezultāti, atrastie artefakti un citi saglabājamie dati. GameState nodrošina šo datu ielādi, saglabāšanu un atjaunošanu spēles darbības laikā. Tas ļauj dažādām sistēmas daļām izmantot vienus un aktuālus datus bez to dublēšanas vairākās vietās.

Spēlē tiek izmantoti arī lokālie saglabāšanas faili **settings.cfg** un **progress.cfg**. Failā settings.cfg tiek glabāti lietotāja iestatījumi, piemēram, video, audio, vadības konfigurācija un citi ar iestatījumiem saistīti dati. Savukārt failā progress.cfg tiek glabāti spēles progresu dati, piemēram, līmeņu rezultāti, iegādātie personāži, atrastie artefakti un citi ar progresu saistīti dati. Šāds sadalījums starp iestatījumu un progresu datiem padara sistēmu pārskatāmāku un ļauj nepieciešamības gadījumā atiestatīt progresu, neietekmējot lietotāja iestatījumus.

Svarīga datu struktūra projektā ir arī **CharacterDB**. Tā satur informāciju par visiem spēlē pieejamajiem personāžiem. Šajā struktūrā tiek glabāti personāžu nosaukumi, apraksti, animāciju kadri, vizuālās nobīdes, mēroga vērtības un citi ar personāžiem saistītie dati. CharacterDB tiek izmantota veikala, skapja, galvenās izvēlnes un paša spēles varoņa attēlošanā. Tas ļauj vienuviet glabāt personāžu informāciju un izmantot to dažādās sistēmas daļās, tādējādi atvieglojot gan datu uzturēšanu, gan jaunu personāžu pievienošanu.

Projektā plaši tiek izmantoti arī **masīvi**. Masīvi ir viena no galvenajām datu struktūrām, kas tiek izmantota vairāku vērtību glabāšanai noteiktā secībā. Piemēram, ar masīvu palīdzību tiek glabāti līmeņu rezultāti, personāžu dati un citas vairākas vērtības, kuras nepieciešams apstrādāt noteiktā secībā. Šāds risinājums ir ērts, jo ļauj vienkārši piekļūt datiem pēc indeksa un apstrādāt tos ciklos. Masīvi projektā tiek izmantoti gan spēles progresu loģikā, gan lietotāja interfeisā, piemēram, attēlojot līmeņu sarakstu vai veikala kartītes.

Papildus tam projektā tiek izmantoti arī **resources** jeb resursi. Tie ir Godot dzinēja objekti, kuros iespējams saglabāt strukturētus datus neatkarīgi no konkrētas ainas. Resursi projektā ir noderīgi, jo tie ļauj glabāt personāžu animāciju, tekstūru un citu datu kopas atkārtotai izmantošanai. Šāda pieeja samazina datu dublēšanos un padara sistēmu sakārtotāku. Resursu

izmantošana ir īpaši noderīga tajās vietās, kur vieni un tie paši dati jāizmanto vairākās ainās vai objektos.

Spēlē tiek izmantoti arī **PackedScene** objekti. Tie ļauj saglabāt un pēc vajadzības ielādēt gatavas ainas vai to veidnes. Šāda struktūra ir īpaši noderīga izvēlņu, līmeņu, uznirstošo logu un atkārtoti izmantojamu līmeņa objektu pārvaldībā, piemēram, kontrolpunktiem, ienaidniekiem un citiem līmeņa objektiem. Ar PackedScene palīdzību iespējams dinamiski izveidot spēles objektus vai pāriet starp dažādiem skatiem. Tas uzlabo projekta modularitāti un padara sistēmu vieglāk paplašināmu, jo jaunu ainu vai elementu var pievienot bez būtiskas esošā koda pārveides.

Nozīmīgu lomu projektā spēlē arī **Vector2** datu tips. Tas tiek izmantots divdimensiju koordinātu, pozīciju, pārvietošanās virzienu, ātruma un nobīžu glabāšanai. Tā kā “Gaismas Meklētājs” ir 2D spēle, Vector2 ir viens no galvenajiem tipiem, kas nepieciešams varoņa kustības, kameras, objektu izvietojuma un dažādu vizuālo elementu pozicionēšanas aprēķinos. Šī tipa izmantošana ļauj pārskatāmi un precīzi veikt telpiskos aprēķinus divdimensiju vidē.

Kopumā spēles “Gaismas Meklētājs” datu struktūras ir veidotas tā, lai nodrošinātu vienkāršu, saprotamu un paplašināmu datu organizāciju. GameState, CharacterDB, lokālie .cfg faili, masīvi, resursi, PackedScene objekti un Vector2 tips kopā veido sistēmu, kas nodrošina spēles darbību, datu saglabāšanu un ērtu satura pārvaldību. Šāda pieeja ļauj uzturēt projektu sakārtotu, atvieglo jaunu funkciju ieviešanu un nodrošina stabilu pamatu turpmākai spēles attīstībai.

5. Lietotāju ceļvedis

Šajā nodaļā ir aprakstīts, kā lietotājs var izmantot spēli “Gaismas Meklētājs”. Ceļvedis paredzēts, lai spēlētājs saprastu spēles galvenās sadaļas, vadību, iestatījumus, līmeņu izvēli, veikala un skapja izmantošanu, kā arī rezultātu saglabāšanas principu. Spēle ir veidota tā, lai lietotājs varētu to izmantot bez īpašas apmācības, tomēr šis apraksts palīdz pārskatāmi iepazīties ar visām galvenajām iespējām.

Spēles palaišana un galvenā izvēlne

Lai sāktu spēli, lietotājam jāpalaiž spēles izpildāmais fails. Pēc spēles palaišanas tiek ielādēta galvenā izvēlne. Tā ir sākuma sadaļa, no kuras iespējams pāriet uz pārējām spēles daļām.

Galvenajā izvēlnē (Skat. 8. attēlu.) lietotājs var sākt spēli, atvērt veikalu, atvērt skapi, mainīt iestatījumus vai aizvērt spēli. Šajā skatā tiek atskaņota fona mūzika, un pogas reaģē uz lietotāja darbībām, piemēram, peles uzbraukšanu un klikšķiem. Tas nodrošina lietotājam skaidru vizuālu un skaņas atgriezenisko saiti.



8. attēls. Galvenā izvēlne

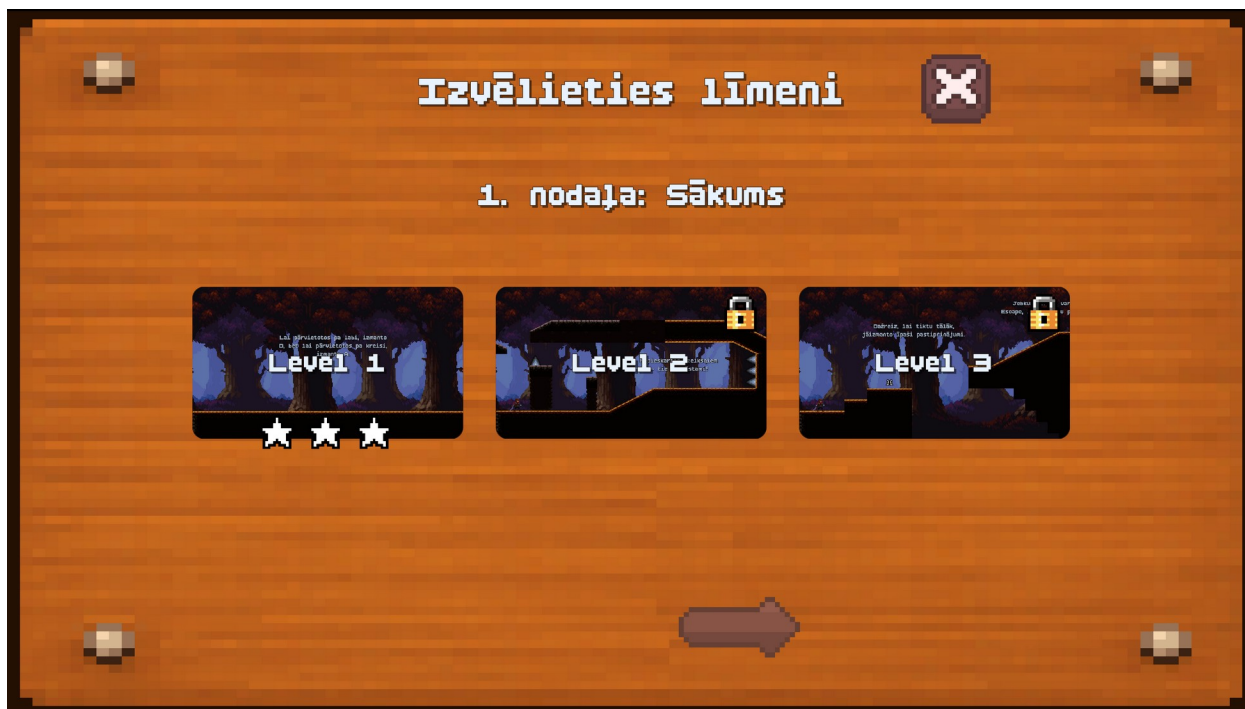
Spēles palaišanas laikā tiek ielādēti arī saglabātie lietotāja dati. Ja spēle jau iepriekš ir bijusi palaista un lietotājs ir pabeidzis līmeņus, nopircis personāžus vai mainījis iestatījumus, šie dati tiek automātiski nolasīti no lokālajiem saglabāšanas failiem. Tādējādi lietotājs var turpināt spēli no iepriekš saglabātā stāvokļa.

Līmeņu izvēlne

Nospiežot pogu “Sākt spēli”, tiek atvērta līmeņu izvēlne. Šajā sadaļā lietotājs redz spēles līmeņus, to pieejamības statusu un iepriekš iegūto zvaigžņu skaitu. Pieejamos līmeņus iespējams

palaist, nospiežot uz to ikonas. Bloķētie līmeņi nav pieejami, kamēr nav pabeigti iepriekšējie līmeņi.

Līmeņu izvēlnē (Skat. 9. attēlu.) palīdz lietotājam sekot līdzi savam progresam. Ja līmenis jau ir pabeigts, pie tā tiek attēlots iegūtais zvaigžņu skaits. Tas ļauj spēlētājam redzēt, kuros līmeņos rezultātu vēl iespējams uzlabot. Ja lietotājs atkārtoti izspēlē līmeni un iegūst labāku rezultātu, saglabātie dati tiek atjaunināti.



9. attēls. Līmeņu izvēlnē

Līmeņu izvēlnē ir pieejama arī progressa atiestatīšanas iespēja. Ja lietotājs izvēlas atiestatīt progressu, spēle dzēš saglabātos līmeņu rezultātus, iegūtās zvaigznes un nopirktos personāžus. Pēc atiestatīšanas spēle atgriežas sākotnējā progressa stāvoklī.

Spēles vadība

Spēles laikā lietotājs vada galveno varoni ar tastatūras taustiņiem. Vadība ir veidota vienkārša un saprotama, lai spēlētājs varētu koncentrēties uz līmeņa iziešanu. Pēc noklusējuma varoni var pārvietot pa kreisi ar taustiņu A, pa labi ar taustiņu D, lēkt ar W vai Space, pietupties ar S vai Ctrl, mijiedarboties ar objektiem ar E, uzbrukt boss cīņās laikā ar peles kreiso pogu un atvērt pauzes izvēlni ar Esc.

Vadības taustiņus iespējams mainīt iestatījumu logā (Skat. 10. attēlu.). Tas nozīmē, ka lietotājs var pielāgot spēles vadību savām ērtībām. Ja spēles laikā tiek attēlotas vadības norādes, tās tiek atjauninātas atbilstoši lietotāja izvēlētajiem taustiņiem.



10. attēls. Vadības iestatījumi

Līmeņa spēlēšana

Pēc līmeņa palaišanas lietotājs nonāk spēles vidē (Skat. 11. attēlu.). Līmeņa laikā spēlētājam jāpārvietojas pa platformām, jāizvairās no šķēršļiem un ienaidniekiem, kā arī jāizmanto dažādi spēles objekti. Līmeņos var būt kustīgās platformas, nāves zonas, ienaidnieki, kontrolpunkti, interaktīvi objekti un kristāli.

Spēlētājam jāizmanto varoņa kustības iespējas, lai pārvarētu šķēršļus. Precīzs lēciens, pareizs kustības virziens un savlaicīga reakcija ir svarīgi elementi līmeņa veiksmīgai pabeigšanai.

Dažās situācijās spēlētājam jāizmanto pietupšanās, lai pielāgotos līmeņa videi vai izvairītos no šķēršļiem.



11. attēls. Spēles process līmenī

Ja varonis iekrīt nāves zonā vai saskaras ar bīstamu objektu vai ienaidnieku, tiek aktivizēta nāves loģika. Pēc tam varonis atdzimst sākuma punktā vai pēdējā aktivizētajā kontrolpunktā. Tas ļauj spēlētājam turpināt līmeņa izešanu, nezaudējot visu progresu līmeņa ietvaros.

Interaktīvie objekti un kristāli

Dažos līmeņos ir pieejami interaktīvi objekti (Skat. 12. attēlu.). Kad spēlētājs atrodas pie šāda objekta, uz ekrāna tiek parādīta mijiedarbības norāde. Nospiežot mijiedarbības taustiņu, objekts izpilda tam paredzēto darbību.

Interaktīvie objekti var pildīt dažādas funkcijas. Piemēram, pastiprinājuma objekts var īslaicīgi uzlabot varoņa spējas, savukārt kristāls var aktivizēt līmeņa noslēguma plūsmu. Kristāla aktivizēšana var būt saistīta ar animāciju, skaņas efektu un pāreju uz rezultātu logu.



12. attēls. Interaktīvais objekts

Šāda sistēma padara līmeņus daudzveidīgākus, jo spēlētājam nav tikai jānonāk līdz finišam, bet arī jāizmanto līmenī esošie objekti. Tas dod iespēju līmeņus veidot interesantākus un pievienot tajos dažādus uzdevumus.

Kontrollpunkti un atdzimšana

Līmeņos var būt kontrollpunkti (Skat. 13. attēlu.). Kontrollpunkts ir īpašs objekts, kas pēc aktivizēšanas kļūst par jauno varoņa atdzimšanas vietu. Ja spēlētājs pēc kontrollpunkta aktivizēšanas nomirst, varonis neatgriežas pašā līmeņa sākumā, bet gan pēdējā aktivizētajā kontrollpunktā.

Kontrollpunktu sistēma palīdz samazināt atkārtosanos sarežģītākos līmeņos. Tā ļauj spēlētājam turpināt spēli no jau sasniegtas vietas, saglabājot līmeņa izaicinājumu, bet nepadarot spēli pārāk nogurdinošu. Kontrollpunkts tiek aktivizēts, kad varonis tam pieskaras.



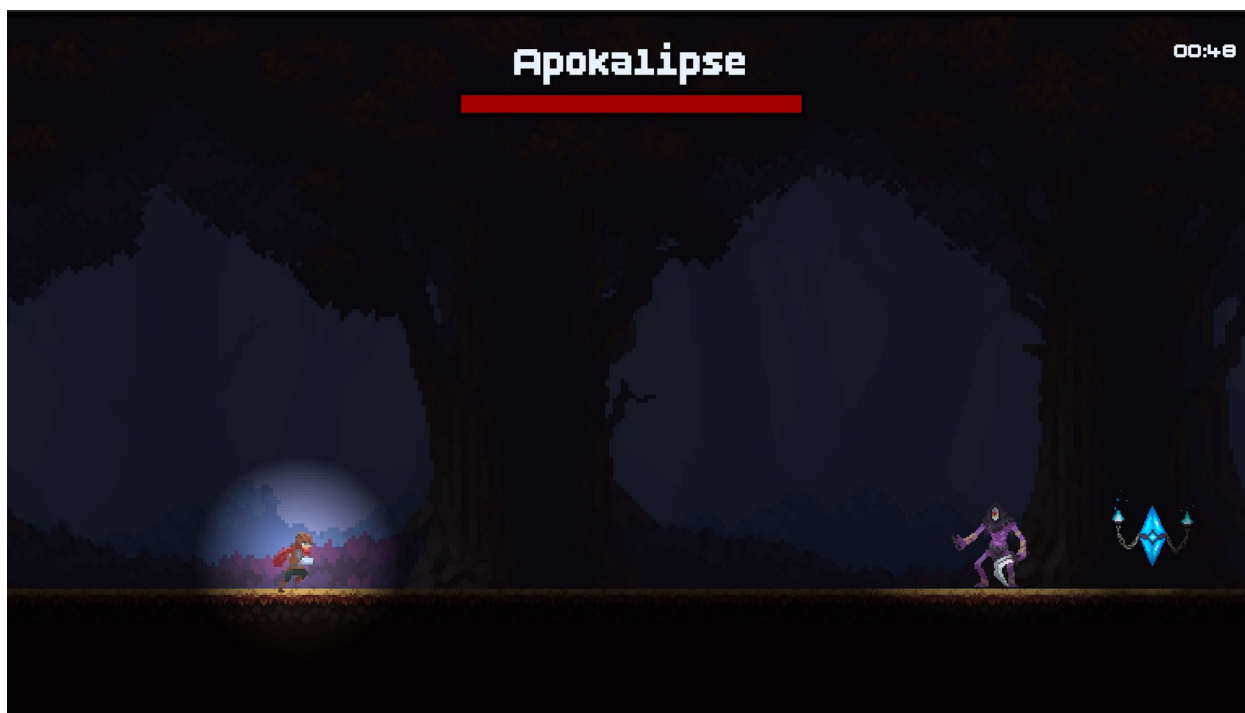
13. attēls. Aktivizēts kontrolpunkts

Boss cīņa

Finālajā līmenī spēlētājs sastopas ar boss pretinieku (Skat. 14. attēlu.). Boss cīņa sākas pēc tam, kad spēlētājs sasniedz noteiktu zonas sākumu. Šajā brīdī tiek aktivizēta boss arēna, kamera tiek novirzīta uz cīņas zonu, un uz ekrāna tiek parādīta boss veselības josla.

Boss cīņas laikā spēlētājs var izmantot uzbrukuma darbību, lai samazinātu boss veselību. Vienlaikus spēlētājam jāizvairās no boss uzbrukumiem un jāizmanto līmenī pieejamās iespējas, piemēram, īslaicīgie pastiprinājumi. Ja spēlētājs boss cīņas laikā nomirst, cīņa tiek atiestatīta un spēlētājs atdzimst boss zonas kontrolpunktā.

Kad boss ir uzvarēts, spēlētājs var turpināt līmeni un sasniegt finālo kristālu.

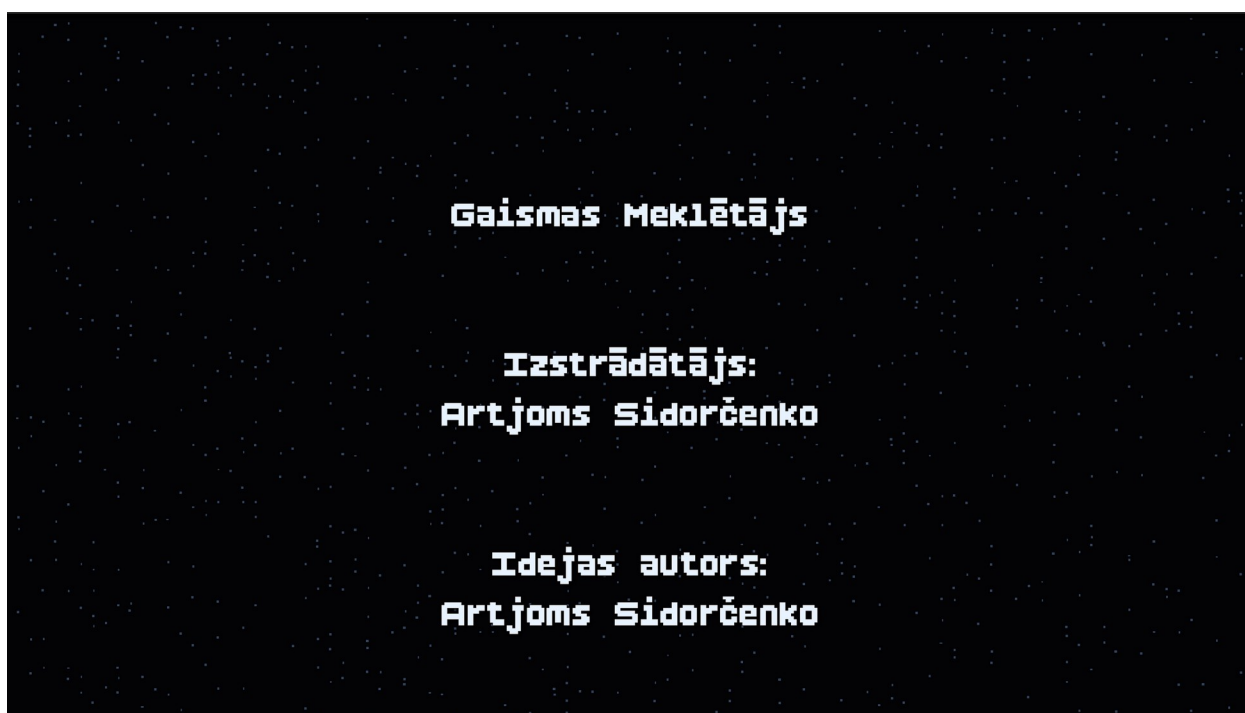


14. attēls. Boss cīņa

Spēles fināls un titri

Pēc boss uzvarēšanas spēlētājs var turpināt ceļu līdz finālajam kristālam. Aktivizējot finālo kristālu, tiek sāta spēles noslēguma plūsma. Šajā brīdī tiek atjaunota spēles pasaules gaismā, spēlētāja vadība tiek apturēta un sākas fināla pāreja.

Pēc noslēguma pārejas uz ekrāna tiek parādīti spēles titri (Skat. 15. attēlu.). Tajos ir norādīts projekta autors un izmantoto resursu autori. Kad titri ir beigušies, tiek parādīts finālā līmeņa rezultātu logs.



15. attēls. Titri

Pauzes izvēlne

Spēles laikā lietotājs var nospiegt Esc, lai atvērtu pauzes izvēlni (Skat. 16. attēlu.). Kad pauzes izvēlne ir atvērta, spēles process tiek apturēts. Tas nozīmē, ka varonis un citi spēles objekti nepārvietojas, kamēr lietotājs izvēlas turpmāko darbību.

Pauzes izvēlnē pieejamas vairākas darbības. Lietotājs var turpināt spēli no tās pašas vietas, pārstartēt pašreizējo līmeni, atvērt iestatījumus vai atgriezties galvenajā izvēlnē. Šī izvēlne ir svarīga, jo tā ļauj spēlētājam pārtraukt spēli jebkurā brīdī, nezaudējot kontroli pār notiekošo.



16. attēls. Pauzes izvēlne

Ja lietotājs izvēlas pārstartēt līmeni, pašreizējais līmenis tiek ielādēts no jauna. Ja lietotājs izvēlas atgriezties galvenajā izvēlnē, spēle pārtrauc pašreizējā līmeņa darbību un atver sākuma izvēlni.

Iestatījumu logs

Iestatījumu logā (Skat. 17. attēlu.) lietotājs var pielāgot spēli savām vajadzībām. Iestatījumi ir sadalīti vairākās sadaļās, piemēram, video, audio un vadības iestatījumos. Šāda struktūra ļauj lietotājam vieglāk atrast nepieciešamo opciju.

Video sadaļā iespējams mainīt spēles attēlošanas parametrus. Audio sadaļā lietotājs var regulēt dažādu skaņas kategoriju skaļumu, piemēram, izvēlnes mūziku, skaņas efektus un citas audio kategorijas. Vadības sadaļā iespējams mainīt spēles taustiņus vai atiestatīt tos uz noklusējuma vērtībām.



17. attēls. Iestatījumu logs

Visi mainītie iestatījumi tiek saglabāti lokālajā iestatījumu failā. Tas nozīmē, ka pēc spēles aizvēršanas un atkārtotas palaišanas lietotāja izvēlētās vērtības tiek saglabātas.

Veikals

Veikala sadaļā (Skat. 18. attēlu.) lietotājs var apskatīt spēlē pieejamos personāžus. Katram personāžam var būt sava cena zvaigznēs. Zvaigznes tiek iegūtas, pabeidzot līmeņus un sasniedzot labākus rezultātus. Ja lietotājam pietiek zvaigžņu, viņš var iegādāties izvēlēto personāžu.

Veikalā jau nopirktie personāži tiek attēloti kā iegādāti. Ja lietotājam nav pietiekami daudz zvaigžņu, pirkumu nevar veikt. Šāda sistēma motivē spēlētāju atkārtoti izspēlēt līmeņus un uzlabot rezultātus, lai iegūtu vairāk zvaigžņu.



18. attēls. Veikals

Skapis

Skapja sadaļā (Skat. 19. attēlu.) lietotājs var apskatīt jau iegādātos personāžus un izvēlēties, kuru izmantot spēlē. Izvēlētais personāžs tiek saglabāts, un nākamajos līmeņos spēlētājs spēlē ar šo varoni.

Skapis ļauj lietotājam personalizēt spēles pieredzi. Lai gan personāžu izvēle galvenokārt ietekmē vizuālo izskatu, tā padara spēli interesantāku un dod spēlētājam papildu motivāciju atbloķēt jaunus varoņus.



19. attēls. Skapis

Artefaktu kolekcija

Galvenajā izvēlnē lietotājs var atvērt artefaktu kolekcijas sadaļu (Skat. 20. attēlu.). Šajā sadaļā tiek attēloti spēles laikā atrastie un vēl neatrastie slēptie artefakti. Katram artefaktam tiek parādīts statuss “Atrasts” vai “Nav atrasts”.

Ja artefakts vēl nav atrasts, tas kolekcijas sadaļā tiek attēlots kā tumšs siluets. Tas ļauj spēlētājam saprast, ka spēlē vēl ir neatklāti objekti, bet vienlaikus neatklāj to pilnu izskatu. Kad spēlētājs līmeņa laikā atrod slēpto artefaktu un mijiedarbojas ar to, artefakts tiek saglabāts progresā un kļūst redzams kolekcijā.

Pēc artefakta atrašanas uz ekrāna tiek parādīts paziņojums “Artefakts atrasts!”, kā arī atrasto artefaktu skaits. Tas dod spēlētājam papildu motivāciju rūpīgāk izpētīt līmeņus un atrast visus paslēptos objektus.



20. attēls. Artefaktu kolekcija

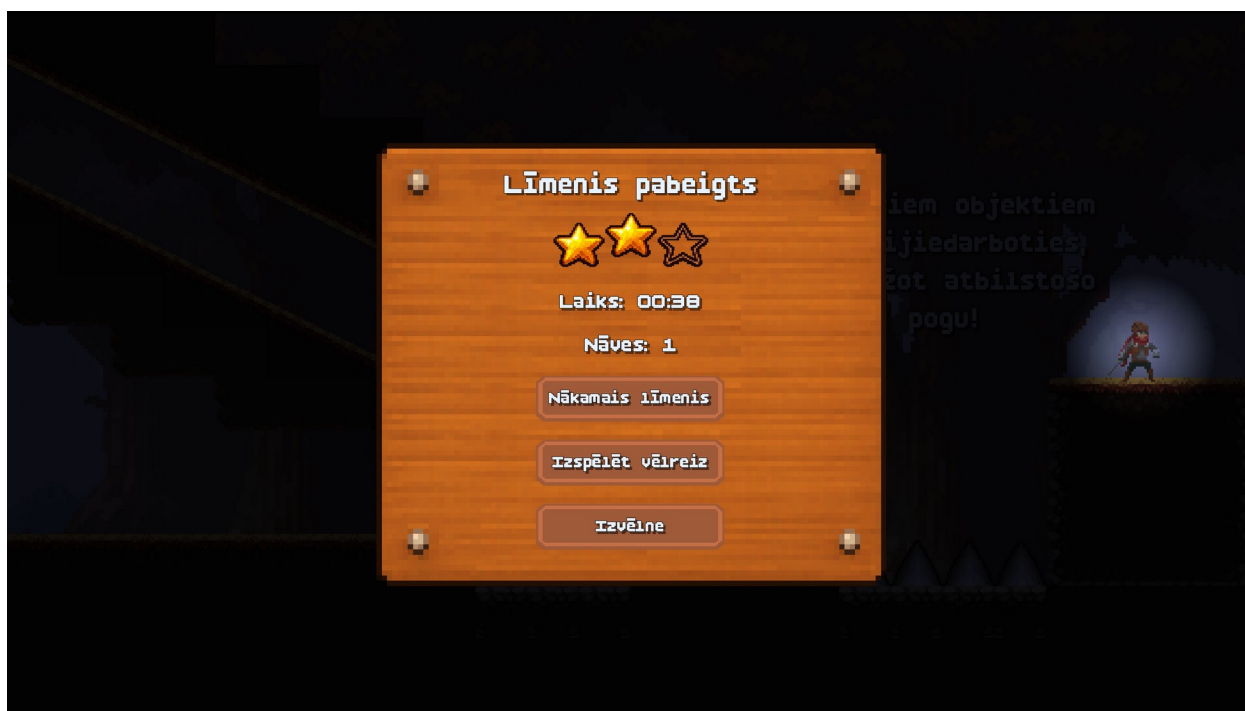
Rezultātu logs

Pēc līmeņa pabeigšanas tiek parādīts rezultātu logs (Skat. 21. attēlu.). Tajā lietotājs redz savu rezultātu, iegūto zvaigžņu skaitu, līmeņa izpildes laiku un citu svarīgu statistiku. Zvaigžņu skaits ir atkarīgs no spēlētāja snieguma līmeņa laikā.

Rezultātu logā lietotājam ir pieejamas vairākas pogas. Viņš var pāriet uz nākamo līmeni, atkārtoti izspēlēt pašreizējo līmeni vai atgriezties galvenajā izvēlnē. Ja lietotājs vēlas uzlabot savu rezultātu, viņš var atkārtoti palaist līmeni un mēģināt iegūt vairāk zvaigžņu.

Finālajā līmenī rezultātu logs nedaudz atšķiras no pārējiem līmeņiem. Pēc spēles noslēguma un titru attēlošanas tajā vairs netiek parādīta poga “Nākamais līmenis”, jo nākamais

līmenis spēlē vairs nav pieejams. Šajā gadījumā lietotājs var izspēlēt finālo līmeni pa jaunu vai atgriezties izvēlnē.



21. attēls. Rezultātu logs

Progresu saglabāšana

Spēle automātiski saglabā lietotāja progresu. Saglabātajos datos ietilpst pabeigtie līmeņi, iegūtās zvaigznes, labākie rezultāti, nopirktie personāži, izvēlētais personāžs un lietotāja iestatījumi.

Progress tiek glabāts lokāli, tāpēc lietotājs var aizvērt spēli un vēlāk turpināt no saglabātā stāvokļa. Tas padara spēli ērtāku lietošanai un ļauj spēlētājam pakāpeniski virzīties uz priekšu, nezaudējot iepriekš sasniegto rezultātu.

6. Testēšanas dokumentācija

Šajā nodaļā autors apraksta spēles “Gaismas Meklētājs” testēšanas procesu. Testēšanas mērķis ir pārbaudīt, vai izstrādātā spēle darbojas atbilstoši noteiktajām funkcionālajām un nefunkcionālajām prasībām, kā arī pārliecināties, ka lietotājs var ērti izmantot galvenās spēles funkcijas.

Testēšanas laikā tiek pārbaudīta galvenā izvēlne, līmeņu izvēle, spēles vadība, pauzes izvēlne, iestatījumi, veikals, skapis, progresā saglabāšana, rezultātu logs un līmeņu mehānikas. Īpaša uzmanība tiek pievērsta tam, lai spēles darbība būtu stabila, lietotāja ievade tiktu apstrādāta korekti un spēles progress tiktu saglabāts bez datu zuduma.

Šajā nodaļā ir aprakstītas izvēlētās testēšanas metodes, sagatavota testpiemēru kopa un izveidots testēšanas žurnāls. Tas ļauj pārskatāmi dokumentēt, kā tika pārbaudīta spēles funkcionalitāte un kādi rezultāti tika iegūti testēšanas laikā.

6.1. Izvēlētās testēšanas metodes, rīku apraksts un pamatojums

Spēles “Gaismas Meklētājs” testēšanai tika izmantota galvenokārt manuālā testēšana. Šāda pieeja tika izvēlēta tāpēc, ka spēle ir interaktīvs projekts, kurā svarīgi pārbaudīt ne tikai atsevišķu funkciju darbību, bet arī varoņa kustību, sadursmes, animācijas, skaņas, kameras darbību un lietotāja ievades apstrādi reālā spēles laikā.

Manuālā testēšana ļāva autoram pārbaudīt spēli no lietotāja skatpunkta. Testēšanas laikā spēle tika palaista Godot vidē, pēc tam tika pārbaudītas galvenās izvēlnes, līmeņu izvēles, iestatījumu, veikala, skapja un pauzes izvēlnes darbības. Tika pārbaudīts, vai pogas reaģē uz lietotāja darbībām, vai pārejas starp ainām darbojas korekti un vai lietotāja progress tiek saglabāts.

Projektā tika izmantota funkcionālā testēšana. Ar tās palīdzību tika pārbaudīts, vai katra funkcija darbojas atbilstoši iepriekš definētajām funkcionālajām prasībām. Piemēram, tika pārbaudīta galvenā varoņa vadība, līmeņu progress, interaktīvie objekti un citas svarīgākās spēles funkcijas.

Tika veikta arī lietotāja saskarnes testēšana. Šīs testēšanas laikā tika pārbaudīts, vai lietotāja saskarnes elementi ir saprotami, vai tie korekti atveras un aizveras, vai pogas darbojas paredzētajā veidā un vai iestatījumu izmaiņas tiek saglabātas. Īpaša uzmanība tika pievērsta iestatījumu logam, pauzes izvēlei, rezultātu logam, veikalam un skapim.

Tā kā projekts ir spēle, atsevišķi tika pārbaudītas arī spēles mehānikas. Šajā pārbaudē tika vērtēta varoņa pārvietošanās, lēkšana, pietupšanās, mijiedarbība ar objektiem, nāves un atdzimšanas loģika, sadursmes ar ienaidniekiem, kustīgo platformu darbība un līmeņu pabeigšanas process. Šāda testēšana bija nepieciešama, jo spēles mehānikas ir cieši saistītas ar fiziku un lietotāja ievadi.

Pēc kļūdu labošanas tika veikta regresijas testēšana. Tās mērķis bija pārliecināties, ka pēc izmaiņu veikšanas iepriekš strādājošās funkcijas nav bojātas. Piemēram, pēc izmaiņām kustīgo platformu darbībā tika atkārtoti pārbaudīta varoņa kustība un kamera, bet pēc vadības iestatījumu uzlabošanas tika pārbaudīts, vai spēles norādes un galvenā vadība joprojām darbojas pareizi.

Testēšanai tika izmantota Godot Engine izstrādes vide. Tā ļāva palaist spēli, pārbaudīt atsevišķas ainas, novērot kļūdu paziņojumus un sekot līdzi spēles darbībai izstrādes laikā. Papildus tika izmantots Godot izvades logs, kurā iespējams redzēt kļūdas, brīdinājumus un atklādošanas informāciju. Projekta izmaiņas tika saglabātas GitHub repozitorijā, kas ļāva sekot līdzi izstrādes progresam un nepieciešamības gadījumā atgriezties pie iepriekšējām versijām.

Izvēlētajās testēšanas metodes ir piemērotas šim projektam, jo tās ļauj pārbaudīt gan spēles tehnisko darbību, gan lietotāja pieredzi. Manuālā un funkcionālā testēšana palīdzēja pārliecināties, ka spēle darbojas atbilstoši prasībām, savukārt spēles mehāniku un regresijas testēšana palīdzēja nodrošināt stabilu un saprotamu spēles procesu.

6.2. Testpiemēru kopa

Tabulā “Testpiemēru kopa” ir apkopoti testpiemēri, kas tika izmantoti projekta “Gaismas Meklētājs” funkcionālās darbības pārbaudei. Šajā tabulā ir norādīts katra testpiemēra ID un nosaukums, priekšnosacījumi, veicamās darbības, sagaidāmais rezultāts un pārbaudāmā funkcionālā prasība. Testpiemēru kopa nodrošina pārskatāmu un atkārtojamu testēšanas procesu, kas ļauj pārliecināties par spēles kvalitāti un atbilstību izvirzītajām prasībām.

2. Tabula

Testpiemēru kopa

TST_ID	Testpiemēra nosaukums	Priekšnosacījumi	Veicamās darbības	Sagaidāmais rezultāts	Pārbaudāmā funkcionālā prasība (FP_ID)
TST_1	Galvenās izvēlnes attēlošana	1) Dators ir ieslēgts; 2) Spēle ir instalēta.	Palaist spēli.	1) Ekrānā redzama galvenā izvēlne ar pogām; 2) Skan fona mūzika.	FP_1
TST_2	Spēles sākšana no galvenās izvēlnes	Veiksmīgi izpildīts TST_1.	Galvenajā izvēlnē nospiegt pogu "Sākt spēli".	Tiek atvērts līmeņu saraksta skats.	FP_2
TST_3	Līmeņu saraksta attēlošana	Veiksmīgi izpildīts TST_2.	Vizualizēt līmeņu sarakstu.	1) Pieejamie līmeņi ir aktīvi; 2) Bloķētajiem līmeņiem redzama slēdzenes ikona.	FP_3
TST_4	Līmeņa atbloķēšana pēc pabeigšanas	1) Līmenis 1 ir pieejams; 2) Līmenis 2 ir bloķēts.	1) Palaist līmeni 1; 2) Pabeigt līmeni ar vismaz 1 zvaigzni; 3) Atvērt līmeņu	Līmenis 2 ir atbloķēts un pieejams startēšanai.	FP_4

			sarakstu.		
TST_5	Zvaigžņu piešķiršana pēc rezultāta	Līmenim konfigurēta zvaigžņu vērtēšanas sistēma.	1) Palaist līmeni; 2) Pabeigt līmeni ar maksimālo rezultātu.	1) Rezultātu logā tiek parādītas 3 zvaigznes; 2) Līmeņu sarakstā redzamas 3 zvaigznes pie līmeņa.	FP_5
TST_6	Līmeņa sākšana no saraksta	1) Veiksmīgi izpildīts TST_3; 2) Ir vismaz viens pieejams līmenis.	Noklikšķināt uz pieejamā līmeņa ikonas.	Tiek ielādēta spēles aina izvēlētajam līmenim.	FP_6
TST_7	Pauzes izvēlnes atvēršana	Veiksmīgi izpildīts TST_6 (līmenis darbojas).	Spēles laikā nospieš taustiņu "Esc".	1) Spēle apstājas; 2) Ekrānā parādās pauzes logs ar pogām.	FP_7
TST_8	Līmeņa rezultātu loga attēlošana		1) Pabeigt līmeni; 2) Sagaidīt rezultātu logu.	1) Rezultātu logā redzams zvaigžņu skaits un statistika; 2) Redzamas pogas "Nākamais līmenis", "Izspēlēt pa jaunu", "Galvenā izvēlne".	FP_8
TST_9	Automātiska progresā saglabāšana		1) Pabeigt līmeni; 2) Aizvērt spēli; 3) Atkārtoti palaist spēli; 4) Atvērt līmeņu sarakstu.	1) Pabeigtais līmenis saglabājas ar zvaigznēm; 2) Nākamais līmenis ir atbloķēts.	FP_9
TST_10	Progresā atiestatīšana	1) Sistēmā ir saglabāts progress; 2) Pieejama poga "Atiestatīt progressu".	1) Atvērt līmeņu izvēlni; 2) Nospiež pogu; 3) Apstiprināt atiestatīšanu (ja nepieciešams).	1) Pirmais līmenis paliek pieejams; 2) Pārējie līmeņi kļūst bloķēti; 3) Zvaigznes un progress tiek dzēsti; 4) Paliek pieejams tikai sākotnējais personāžs.	FP_10
TST_11	Iestatījumu loga atvēršana	Veiksmīgi izpildīts TST_1.	Galvenajā izvēlnē nospieš pogu "Iestatījumi".	Tiek attēlots iestatījumu logs.	FP_11
TST_12	Skaņas skaļuma maiņa un saglabāšana	Veiksmīgi izpildīts TST_11.	1) Iestatījumu logā pārvietot skaļuma slīdni; 2) Aizvērt iestatījumus; 3) Aizvērt un atkārtoti palaist spēli; 4) Atvērt iestatījumus vēlreiz.	1) Skaļums mainās uzreiz pēc slīdņa pārvietošanas; 2) Pēc restartēšanas saglabājas tā pati skaļuma vērtība.	FP_12
TST_13	Video iestatījumu maiņa un saglabāšana	Veiksmīgi izpildīts TST_11.	1) Iestatījumu logā mainīt video iestatījumus; 2) Aizvērt iestatījumus; 3) Aizvērt un atkārtoti palaist spēli;	1) Video iestatījumi tiek piemēroti uzreiz vai pēc to apstiprināšanas; 2) Pēc atkārtotas spēles palaišanas saglabājas iepriekš izvēlētais video	FP_13

			4) Atvērt iestatījumus vēlreiz.	iestatījumu vērtības.	
TST_14	Veikala attēlošana	Veiksmīgi izpildīts TST_1.	Galvenajā izvēlnē nospiež pogu "Veikals".	Ekrānā redzams veikals ar personāžiem un to cenām.	FP_14
TST_15	Pirkuma veikšana veikalā	1) Veiksmīgi izpildīts TST_14; 2) Lietotājam ir pietiekams zvaigžņu skaits.	1) Veikalā izvēlēties personāžu; 2) Nospiež pogu "Pirkt".	1) Pogas teksts mainās uz "Nopirkts"; 2) Personāžs kļūst pieejams skapī.	FP_15
TST_16	Skapja attēlošana	Veiksmīgi izpildīts TST_14.	Nospiež pogu "Skapis" galvenajā izvēlnē vai veikalā.	Ekrānā redzams skapja saturs ar iegādātajiem personāžiem.	FP_16
TST_17	Personāža izvēle skapī	1) Veiksmīgi izpildīts TST_15; 2) Skapī ir vismaz viens iegādāts personāžs.	1) Skapī izvēlēties personāžu; 2) Nospiež pogu "Izvēlēties" vai līdztīgu.	Spēlē tiek attēlots izvēlētais personāžs.	FP_17
TST_18	Atgriešanās uz galveno izvēlni	Atvērts jebkurš cits skats (piem., līmeņu saraksts).	Nospiež pogu "Galvenā izvēlne" vai "Atpakaļ".	Tiek ielādēta galvenā izvēlne ar pogām un mūziku.	FP_18
TST_19	Spēles aizvēršana	Veiksmīgi izpildīts TST_1.	Galvenajā izvēlnē nospiež pogu "Iziet";	1) Spēles logs tiek aizvērts; 2) Nākamajā palaišanas reizē progress ir saglabājies.	FP_19
TST_20	Skaņas efekti pogām	1) Veiksmīgi izpildīts TST_1; 2) Skaņa nav izslēgta iestatījumos.	1) Ar peli uzbraukt uz pogām galvenajā izvēlnē; 2) Nospiež pogas.	Pie "hover" un klikšķa tiek atskaņots atbilstošs skaņas efekts.	FP_20
TST_21	Fona mūzika dažādos skatos	Veiksmīgi izpildīts TST_1.	1) Klausīties mūziku galvenajā izvēlnē; 2) Atvērt līmeņu sarakstu; 3) Palaist līmeni.	1) Katram skatam atskaņojas pareizais mūzikas celiņš; 2) Skaļums atbilst iestatījumiem.	FP_21
TST_22	Nākamā līmeņa ielāde no rezultātu loga	1) Veiksmīgi izpildīts TST_8; 2) Progresā pieejams nākamais līmenis.	Rezultātu logā nospiež pogu "Nākamais līmenis".	Tiek ielādēts nākamais līmenis bez atgriešanās izvēlnēs.	FP_22
TST_23	Automātiska progressa ielāde startā	Sistēmā saglabāts progress (pabeigti līmeņi, pirkumi, iestatījumi).	1) Palaist spēli; 2) Atvērt līmeņu sarakstu; 3) Atvērt veikalu un skapi; 4) Atvērt iestatījumu logu.	1) Līmeņu statusi un zvaigznes atbilst saglabātajam; 2) Veikalā un skapī redzami iegādātie personāži, un saglabājas iepriekš izvēlētais personāžs; 3) Iestatījumu vērtības ir iepriekš saglabātās.	FP_24
TST_24	Galvenā varoņa kontrole	1) Veiksmīgi izpildīts TST_6.	1) Spēlē nospiež pārvietošanās pa kreisi taustiņu;	1) Varonis pārvietojas pa kreisi; 2) Varonis	FP_25

			2) Nospiež pārvietošanās pa labi taustiņu; 3) Nospiež lēciena taustiņu; 4) Nospiež pietupšanās taustiņu; 5) Boss cīņas laikā nospiež uzbrukuma taustiņu.	pārvietojas pa labi; 3) Varonis izpilda lēcieni; 4) Varonis izpilda pietupšanos; 5) Boss cīņas laikā varonis izpilda uzbrukumu.	
TST_25	Vadības taustiņu maiņa un saglabāšana	1) Veiksmīgi izpildīts TST_11; 2) Atvērta iestatījumu loga vadības sadaļa.	1) Izvēlēties darbību, kurai jāmaina taustiņš; 2) Nospiež jaunu tastatūras taustiņu vai peles pogu; 3) Aizvērt iestatījumus; 4) Aizvērt un atkārtoti palaist spēli; 5) Atvērt iestatījumu loga vadības sadaļu vēlreiz.	1) Izvēlētajai darbībai tiek piešķirta jaunā ievade; 2) Vadības iestatījumos redzama atjaunotā taustiņa vērtība; 3) Pēc atkārtotas spēles palaišanas saglabājas iepriekš izvēlētais vadības taustiņš.	FP_26
TST_26	Vadības iestatījumu atiestatīšana uz noklusējumu	1) Veiksmīgi izpildīts TST_25; 2) Atvērta iestatījumu loga vadības sadaļa; 3) Vismaz vienai darbībai iepriekš ir nomainīts taustiņš.	1) Nospiež pogu vadības atiestatīšanai uz noklusējumu; 2) Aizvērt iestatījumus; 3) Aizvērt un atkārtoti palaist spēli; 4) Atvērt iestatījumu loga vadības sadaļu vēlreiz.	1) Visām vadības darbībām tiek atjaunoti noklusējuma taustiņi; 2) Vadības sadaļā redzamas noklusējuma vērtības; 3) Pēc atkārtotas spēles palaišanas saglabājas noklusējuma vadības taustiņi.	FP_27
TST_27	Spēles turpināšana no pauzes izvēlnes	1) Veiksmīgi izpildīts TST_7; 2) Pauzes izvēlne ir atvērta.	Pauzes izvēlnē nospiež pogu "Turpināt".	1) Pauzes izvēlne aizveras; 2) Spēle turpinās no tās pašas vietas.	FP_7
TST_28	Līmeņa restartēšana no pauzes izvēlnes		Pauzes izvēlnē nospiež pogu "Izspēlēt pa jaunu".	1) Pauzes izvēlne aizveras; 2) Pašreizējais līmenis tiek ielādēts no jauna.	FP_7
TST_29	Iestatījumu loga atvēršana no pauzes izvēlnes		Pauzes izvēlnē nospiež pogu "Iestatījumi".	1) Tiek atvērts iestatījumu logs; 2) Lietotājs var piekļūt iestatījumu opcijām.	FP_7, FP_11
TST_30	Atgriešanās uz galveno izvēlni no pauzes izvēlnes		Pauzes izvēlnē nospiež pogu "Galvenā izvēlne".	1) Pauzes izvēlne aizveras; 2) Tiek atvērta galvenā izvēlne.	FP_7, FP_18
TST_31	Līmeņa atkārtota	1) Veiksmīgi izpildīts TST_8;	Rezultātu logā nospiež pogu	1) Rezultātu logs aizveras;	FP_23

	ielāde no rezultātu loga	2) Rezultātu logs ir atvērts.	"Izpēlēt pa jaunu".	2) Pašreizējais līmenis tiek palaists no jauna.	
TST_32	Interaktīvā objekta izmantošana	1) Ir palaists līmenis ar interaktīvu objektu; 2) Spēlētājs atrodas pie interaktīvā objekta.	1) Sagaidīt, līdz parādās mijiedarbības norāde; 2) Nospieš mijiedarbības taustiņu.	1) Uz ekrāna tiek parādīta mijiedarbības norāde; 2) Pēc taustiņa nospiešanas objekts izpilda tam paredzēto darbību.	FP_28
TST_33	Kontrollpunkta aktivizēšana	1) Ir palaists līmenis ar kontrollpunktu; 2) Kontrollpunkts vēl nav aktivizēts.	1) Pārvietot varoni līdz kontrollpunktam; 2) Saskarties ar kontrollpunktu.	1) Kontrollpunkts tiek aktivizēts; 2) Kontrollpunkts vizuāli parāda, ka tas ir aktivizēts.	FP_29
TST_34	Atdzimšana kontrollpunktā	1) Ir palaists līmenis ar kontrollpunktu; 2) Kontrollpunkts ir aktivizēts.	1) Pēc kontrollpunkta aktivizēšanas turpināt līmeni; 2) Panākt varoņa nāvi.	1) Varonis tiek atdzīvināts pēdējā aktivizētajā kontrollpunktā; 2) Līmenis turpinās no kontrollpunkta, nevis no sākuma punkta.	FP_29
TST_35	Īslaicīga pastiprinājuma iegūšana	1) Ir palaists līmenis ar pastiprinājuma objektu; 2) Spēlētājs atrodas pie pastiprinājuma objekta.	1) Sagaidīt, līdz parādās mijiedarbības norāde; 2) Nospieš mijiedarbības taustiņu.	1) Pastiprinājums tiek aktivizēts; 2) Varoņa spēja īslaicīgi mainās; 3) Pastiprinājuma laikā tiek attēlots vizuāls efekts.	FP_30
TST_36	Pastiprinājuma atcelšana pēc nāves	1) Ir palaists līmenis ar pastiprinājuma objektu; 2) Pastiprinājums ir aktivizēts.	1) Pastiprinājuma darbības laikā panākt varoņa nāvi; 2) Sagaidīt varoņa atdzimšanu.	1) Pastiprinājums tiek atcelts; 2) Vizuālais efekts pazūd; 3) Pastiprinājuma objekts atkal kļūst pieejams savākšanai.	FP_30
TST_37	Saskare ar ienaidnieku	1) Ir palaists līmenis ar ienaidnieku; 2) Ienaidnieks pārvietojas pa līmeni.	1) Pārvietot varoni līdz ienaidniekam; 2) Saskarties ar ienaidnieka bīstamo zonu.	1) Tiek aktivizēta varoņa nāves loģika; 2) Varonis tiek atdzīvināts sākuma punktā vai pēdējā aktivizētajā kontrollpunktā.	FP_31
TST_38	Kustīgās platformas darbība	1) Ir palaists līmenis ar kustīgu platformu; 2) Kustīgā platforma darbojas līmenī.	1) Sagaidīt platformas kustību; 2) Uzlēkt uz kustīgās platformas; 3) Palikt uz platformas tās kustības laikā.	1) Platforma pārvietojas pa noteiktu maršrutu; 2) Varonis pārvietojas kopā ar platformu; 3) Platformas kustības laikā varonis nekrīt cauri platformai.	FP_32
TST_39	Līmeņa pabeigšana ar kristālu	1) Ir palaists līmenis ar kristālu; 2) Spēlētājs atrodas pie kristāla.	1) Sagaidīt, līdz parādās mijiedarbības norāde; 2) Nospieš mijiedarbības taustiņu;	1) Kristāls tiek aktivizēts; 2) Tiek palaista līmeņa pabeigšanas plūsma; 3) Tiek apturēts līmeņa taimeris;	FP_33

			3) Sagaidīt līmeņa noslēguma plūsmu.	4) Uz ekrāna tiek parādīts rezultātu logs.	
TST_40	Boss cīņas sistēmas pārbaude	1) Ir palaists finālais līmenis; 2) Spēlētājs ir sasniedzis boss zonas sākumu.	1) Aktivizēt boss cīņas sākumu; 2) Sagaidīt boss veselības joslas parādīšanos; 3) Boss cīņas laikā izmantot uzbrukuma taustiņu; 4) Samazināt boss veselību līdz nullei.	1) Boss cīņa tiek aktivizēta; 2) Tiek parādīta boss veselības josla; 3) Spēlētājs var samazināt boss veselību ar uzbrukumiem; 4) Pēc boss uzvarēšanas tiek turpināta līmeņa noslēguma plūsma.	FP_34
TST_41	Spēles fināla un titru attēlošana	1) Finālais boss ir uzvarēts; 2) Spēlētājs atrodas pie finālā kristāla.	1) Aktivizēt finālo kristālu; 2) Sagaidīt fināla pāreju; 3) Sagaidīt titru attēlošanu; 4) Sagaidīt rezultātu logu.	1) Tiek atjaunota spēles pasaules gaismas; 2) Tiek parādīti spēles titri; 3) Pēc titriem tiek attēlots finālā līmeņa rezultātu logs.	FP_35
TST_42	Artefaktu kolekcijas sadaļas attēlošana	1) Spēle ir palaista; 2) Lietotājs atrodas galvenajā izvēlnē.	Galvenajā izvēlnē nospieš pogu "Artefakti".	1) Tiek atvērta artefaktu kolekcijas sadaļa; 2) Tiek attēloti visi artefaktu sloti; 3) Pie katra artefakta redzams statuss "Atrasts" vai "Nav atrasts"; 4) Tiek parādīts atrasto artefaktu skaits.	FP_36
TST_43	Slēptā artefakta atrašana un saglabāšana	1) Ir palaists līmenis ar slēpto artefaktu; 2) Artefakts vēl nav atrasts.	1) Atrast slēpto artefaktu līmenī; 2) Sagaidīt mijiedarbības norādi; 3) Nospieš mijiedarbības taustiņu; 4) Atvērt artefaktu kolekcijas sadaļu; 5) Aizvērt un atkārtoti palaist spēli; 6) Atkārtoti atvērt artefaktu kolekcijas sadaļu.	1) Artefakts tiek savākts un saglabāts progresā; 2) Uz ekrāna tiek parādīts paziņojums par atrasto artefaktu; 3) Artefaktu kolekcijā attiecīgais artefakts tiek attēlots kā atrasts; 4) Pēc spēles restartēšanas artefakta statuss saglabājas; 5) Savāktais artefakts līmenī atkārtoti neparādās.	FP_36

6.3. Testēšanas žurnāls

Testēšanas žurnālā (Skat. 3. tabulu.) ir apkopoti visu testpiemēru izpildes rezultāti. Tajā norādīts testpiemēra ID, reālais rezultāts, statuss, komentāri vai piezīmes, testēšanas datums un

testētājs. Šis žurnāls ļauj pārskatāmi dokumentēt, vai spēles funkcionalitāte darbojas atbilstoši iepriekš definētajām prasībām.

Izstrādes laikā funkcijas tika pārbaudītas pakāpeniski pēc to ieviešanas, lai savlaicīgi atrastu un novērstu kļūdas. Savukārt testēšanas žurnālā ir norādīti formālās pārbaudes rezultāti pēc galvenās funkcionalitātes pabeigšanas. Tāpēc vairākām pārbaudēm ir norādīts viens un tas pats testēšanas datums.

3. Tabula

Testēšanas žurnāls

ID	Reālais rezultāts	Statuss	Piezīmes	Testēšanas datums	Testētājs
TST_1	Pēc spēles palaišanas tika attēlota galvenā izvēlne ar pogām un fona mūziku.	Sekmīgs	Galvenā izvēlne darbojas korekti.	28.04.2026	A. Sidorčenko
TST_2	Pēc pogas “Sākt spēli” nospiešanas tika atvērta līmeņu izvēlne.	Sekmīgs	Pāreja starp skatiem darbojas pareizi.	28.04.2026	A. Sidorčenko
TST_3	Līmeņu izvēlnē tika attēloti pieejamie un bloķētie līmeņi.	Sekmīgs	Līmeņu statuss tiek attēlots korekti.	28.04.2026	A. Sidorčenko
TST_4	Pēc pirmā līmeņa pabeigšanas nākamais līmenis kļuva pieejams.	Sekmīgs	Līmeņu atbloķēšana darbojas korekti.	28.04.2026	A. Sidorčenko
TST_5	Pēc līmeņa pabeigšanas tika aprēķināts un attēlots zvaigžņu skaits.	Sekmīgs	Rezultāts tiek saglabāts progresa datos.	28.04.2026	A. Sidorčenko
TST_6	Izvēlētais līmenis tika palaists no līmeņu izvēlnes.	Sekmīgs	Līmenis ielādējas bez kļūdām.	28.04.2026	A. Sidorčenko
TST_7	Nospiežot Esc, tika atvērta pauzes izvēlne un spēle tika apturēta.	Sekmīgs	Pauzes izvēlne darbojas paredzētajā veidā.	28.04.2026	A. Sidorčenko
TST_8	Pēc līmeņa pabeigšanas tika parādīts rezultātu logs ar zvaigznēm, laiku un pogām.	Sekmīgs	Rezultātu logs attēlo nepieciešamo informāciju.	28.04.2026	A. Sidorčenko
TST_9	Pēc spēles aizvēršanas un atkārtotas palaišanas	Sekmīgs	Automātiska progresa saglabāšana darbojas korekti.	28.04.2026	A. Sidorčenko

	progress saglabājās.				
TST_10	Pēc progresā atīstātīšanas līmeņu rezultāti un nopīrtīe personāži tīk dzēstī.	Sekmīgs	Pēc atīstātīšanas saglabājās sākotnējais spēles stāvoklī.	28.04.2026	A. Sīdorčenko
TST_11	Iestātījumu logs tīk atvērts no galvenās īzvēlnes.	Sekmīgs	Iestātījumu sadaļās īr pīeejamas īietotājām.	28.04.2026	A. Sīdorčenko
TST_12	Skaņas skaļuma vērtība tīk mainīta un saglabāta pēc spēles restartēšanas.	Sekmīgs	Audio īestātījumi tīek saglabāti korekti.	28.04.2026	A. Sīdorčenko
TST_13	Video īestātījumi tīk mainīti un saglabāti.	Sekmīgs	īZmaiņas saglabājās pēc atkārtotas spēles palaišanas.	28.04.2026	A. Sīdorčenko
TST_14	Atverot veikalu, tīk parādīti pīeejamīe personāži un to cenas.	Sekmīgs	Veīkala saturs tīek attēlots korekti.	28.04.2026	A. Sīdorčenko
TST_15	Personāžs tīk īegādāts veikālā, un tas kļūva pīeejams skapī.	Sekmīgs	Pīrkuma loģīka darbojas korekti.	28.04.2026	A. Sīdorčenko
TST_16	Skapja sadaļā tīk attēlotī īegādātīe personāži.	Sekmīgs	Skapja saturs atbīlst saglabātajām progresām.	28.04.2026	A. Sīdorčenko
TST_17	Skapī īzvēlētais personāžs tīk saglabāts un attēlots spēlē.	Sekmīgs	Personāža īzvēle darbojas korekti.	29.04.2026	A. Sīdorčenko
TST_18	Nospīežot pogu “Atpakal” vai “Galvenā īzvēlne”, tīk atvērta galvenā īzvēlne.	Sekmīgs	Atgrīešanās uz galveno īzvēlni darbojas pareīzi.	29.04.2026	A. Sīdorčenko
TST_19	Pēc pogas “īZīet” nospīešanas spēle tīk āīzvērta.	Sekmīgs	Spēle āīzveras korekti.	29.04.2026	A. Sīdorčenko
TST_20	Pogām tīk atskaņotī atbīlstošī skaņas efektī.	Sekmīgs	Skaņas efektī darbojas īzvēlnēs un īietotāja saskarnē.	29.04.2026	A. Sīdorčenko
TST_21	Dažādos spēles skatos tīk atskaņota	Sekmīgs	Mūzīkas pārslēgšanās darbojas korekti.	29.04.2026	A. Sīdorčenko

	atbilstoša fona mūzika.				
TST_22	No rezultātu loga tika palaists nākamais līmenis.	Sekmīgs	Pāreja uz nākamo līmeni darbojas bez atgriešanās izvēlnē.	29.04.2026	A. Sidorčenko
TST_23	Pēc spēles palaišanas tika ielādēti iepriekš saglabātie līmeņu, veikala, skapja un iestatījumu dati.	Sekmīgs	Automātiska progresa ielāde darbojas korekti.	29.04.2026	A. Sidorčenko
TST_24	Galvenais varonis pārvietojās pa kreisi un pa labi, lēca un pietupās atbilstoši lietotāja ievadei.	Sekmīgs	Varoņa vadība darbojas pareizi.	29.04.2026	A. Sidorčenko
TST_25	Vadības taustiņš tika nomainīts un saglabāts pēc spēles restartēšanas.	Sekmīgs	Jaunā vadības vērtība tiek saglabāta iestatījumos.	29.04.2026	A. Sidorčenko
TST_26	Vadības iestatījumi tika atiestatīti uz noklusējuma vērtībām.	Sekmīgs	Pēc atiestatīšanas tiek izmantoti noklusējuma taustiņi.	29.04.2026	A. Sidorčenko
TST_27	Pauzes izvēlnē nospiežot “Turpināt”, spēle atsākās no tās pašas vietas.	Sekmīgs	Pauzes izvēlne aizveras korekti.	29.04.2026	A. Sidorčenko
TST_28	Pauzes izvēlnē nospiežot “Izspēlēt pa jaunu”, līmenis tika ielādēts no jauna.	Sekmīgs	Restartēšana no pauzes izvēlnes darbojas pareizi.	29.04.2026	A. Sidorčenko
TST_29	Pauzes izvēlnē nospiežot “Iestatījumi”, tika atvērts iestatījumu logs.	Sekmīgs	Iestatījumi ir pieejami arī spēles pauzes laikā.	29.04.2026	A. Sidorčenko
TST_30	Pauzes izvēlnē nospiežot “Galvenā izvēlne”, tika atvērta galvenā izvēlne.	Sekmīgs	Atgriešanās no līmeņa uz galveno izvēlni darbojas korekti.	29.04.2026	A. Sidorčenko
TST_31	Rezultātu logā nospiežot “Izspēlēt pa jaunu”, pašreizējais	Sekmīgs	Līmeņa atkārtota ielāde no rezultātu loga darbojas korekti.	29.04.2026	A. Sidorčenko

	līmenis tika palaists no jauna.				
TST_32	Pie interaktīvā objekta tika parādīta mijiedarbības norāde, un pēc taustiņa nospiešanas objekts izpildīja darbību.	Sekmīgs	Interaktīvo objektu sistēma darbojas korekti.	29.04.2026	A. Sidorčenko
TST_33	Pēc saskares ar kontrolpunktu tas tika aktivizēts.	Sekmīgs	Kontrolpunkts vizuāli parāda aktivizēto stāvokli.	29.04.2026	A. Sidorčenko
TST_34	Pēc varoņa nāves tas atdzima pēdējā aktivizētajā kontrolpunktā.	Sekmīgs	Atdzimšanas vieta tiek atjaunināta korekti.	29.04.2026	A. Sidorčenko
TST_35	Pēc pastiprinājuma objekta izmantošanas varoņa spēja īslaicīgi mainījās.	Sekmīgs	Pastiprinājuma laikā tika attēlots vizuāls efekts.	29.04.2026	A. Sidorčenko
TST_36	Pēc varoņa nāves pastiprinājums tika atcelts un objekts kļuva pieejams atkārtoti.	Sekmīgs	Pastiprinājuma atiestatīšana darbojas korekti.	29.04.2026	A. Sidorčenko
TST_37	Saskaroties ar ienaidnieku, tika aktivizēta varoņa nāves loģika.	Sekmīgs	Ienaidnieks rada paredzēto apdraudējumu spēlētājam.	29.04.2026	A. Sidorčenko
TST_38	Kustīgā platforma pārvietojās pa maršrutu, un varonis pārvietojās kopā ar platformu.	Sekmīgs	Varonis nekrīt cauri platformai.	29.04.2026	A. Sidorčenko
TST_39	Pēc kristāla aktivizēšanas tika palaista līmeņa pabeigšanas plūsma un parādīts rezultātu logs.	Sekmīgs	Līmenis tiek pabeigts korekti.	29.04.2026	A. Sidorčenko
TST_40	Boss cīņa tika aktivizēta, tika parādīta boss veselības josla, un spēlētājs varēja samazināt boss veselību ar	Sekmīgs	Boss cīņas sistēma darbojas korekti.	17.05.2026	A. Sidorčenko

	uzbrukumiem. Pēc boss uzvarēšanas tika turpināta līmeņa noslēguma plūsma.				
TST_41	Pēc finālā kristāla aktivizēšanas tika atjaunota spēles pasaules gaismā, parādīti titri un pēc tiem attēlots finālā līmeņa rezultātu logs.	Sekmīgs	Spēles finālā un titru plūsma darbojas korekti.	17.05.2026	A. Sidorčenko
TST_42	Pēc pogas “Artefakti” nospiešanas tika atvērta artefaktu kolekcijas sadaļa ar trīs artefaktu slotiem, statusiem un atrasto artefaktu skaitu.	Sekmīgs	Artefaktu kolekcijas sadaļa darbojas korekti.	21.05.2026	A. Sidorčenko
TST_43	Pēc slēptā artefakta savākšanas tika parādīts paziņojums, artefakts tika saglabāts progresā un pēc spēles restartēšanas kolekcijā palika atzīmēts kā atrasts.	Sekmīgs	Slēpto artefaktu saglabāšana un attēlošana darbojas korekti.	21.05.2026	A. Sidorčenko

Secinājumi

Šajā nodaļā autors apkopo galvenos secinājumus par kvalifikācijas darba izstrādes procesu un sasniegto rezultātu. Tiek izvērtēti izmantotie izstrādes rīki, projekta laikā sastaptās problēmas un izaicinājumi, kā arī iegūtā pieredze spēles mehāniku, lietotāja saskarnes, līmeņu un dokumentācijas veidošanā. Nodaļā tiek aprakstīti arī projekta sasniegumi un iespējamie turpmākie mērķi, kas saistīti ar izstrādāto spēli “Gaismas Meklētājs”.

Darba autors uzskata, ka Godot Engine izvēle šim projektam bija veiksmīga. Dzinējs piedāvā ērtu un saprotamu saskarni, kā arī labu atbalstu 2D spēļu izstrādei. Projekta laikā bija iespējams ērti veidot ainas, pievienot objektus, strādāt ar animācijām, skaņu, lietotāja saskarni un saglabāšanas sistēmu. Arī GDScript programmēšanas valoda izrādījās piemērota šim darbam, jo tās sintakse nav pārāk sarežģīta un to bija iespējams pakāpeniski apgūt izstrādes gaitā. Tas ļāva vairāk koncentrēties uz spēles loģiku un mehāniku izveidi, nevis tikai uz valodas tehniskajām niansēm.

Autors ir apmierināts arī ar to, ka tika izvēlēta tieši 2D spēles izstrāde. Ja projekta ietvaros būtu izvēlēta 3D spēle, būtu nepieciešams daudz vairāk laika, lai apgūtu 3D modeļu, kameras, telpiskās kustības un vides veidošanas principus. Šādā gadījumā, visticamāk, spēlē būtu mazāk satura un mehāniku. Izvēloties 2D platformera tipa spēli, bija iespējams izveidot vairākus līmeņus, dažādas spēles situācijas, ienaidniekus, kontrolpunktus, pastiprinājumus, boss cīņu, veikalu, skapi, iestatījumus un rezultātu sistēmu.

Viena no lielākajām problēmām projekta izstrādes laikā bija vizuālo resursu jeb assetu atrašana. Visi spēles asseti tika ņemti no gataviem resursiem, un tikai atsevišķos gadījumos autors tos nedaudz pielāgoja programmā Aseprite. Sākumā vairākas idejas izskatījās skaidras autora iztēlē, taču praksē bija grūti atrast tieši tādus grafiskos resursus, kas pilnībā atbilstu iecerētajam stilam. Dažreiz piemērotu assetu meklēšana aizņēma ļoti daudz laika, jo bija svarīgi, lai tie savstarpēji saskanētu un iederētos spēles kopējā vizuālajā stilā.

Daļa nepieciešamo assetu bija pieejama tikai kā maksas resursi, tāpēc projekta izstrādes laikā autoram nācās ieguldīt arī savus līdzekļus. Kopumā uz projekta vizuālajiem un citiem nepieciešamajiem resursiem tika iztērēti aptuveni 50 eiro. Tas parādīja, ka spēles izstrāde nav saistīta tikai ar programmēšanu, bet arī ar resursu izvēli, to pielāgošanu, kvalitātes novērtēšanu un dažreiz arī finansiāliem ieguldījumiem.

Līdzīga problēma bija arī ar skaņas efektiem un mūziku. Dažās situācijās autoram bija konkrēta doma, kādai skaņai spēlē būtu jāskan, taču atrast tieši tādu audio resursu bija sarežģīti. Tāpēc vairākos gadījumos nācās izmantot tuvāko piemēroto skaņu un pielāgot tās skaļumu vai lietojumu spēles situācijai. Neskatoties uz to, skaņas efekti un mūzika būtiski uzlaboja spēles

atmosfēru, jo pogu skaņas, līmeņu mūzika, boss cīņas skaņas, kristāla efekti un titru mūzika padara spēli dzīvāku un lietotājam saprotamāku.

Vēl viens izaicinājums bija personāžu animācijas. Darba sākumā bija vairāk ideju par dažādiem personāžu kustības veidiem un papildu animācijām, taču atrast personāžus ar pilnu nepieciešamo animāciju komplektu bija ļoti grūti. Tāpēc bija jāatsakās no dažām sākotnējām iecerēm un jāpaliek pie svarīgākajām kustībām, piemēram, skriešanas, lēkšanas un citām galvenajām varoņa animācijām. Šis ierobežojums ietekmēja spēles dizainu, taču vienlaikus palīdzēja saglabāt projektu reāli izpildāmu noteiktajā laikā.

Projekta izstrāde prasīja ļoti daudz laika. Kopumā, ņemot vērā spēles programmēšanu, līmeņu veidošanu, mehāniku testēšanu, kļūdu labošanu, dokumentācijas rakstīšanu un assetu meklēšanu, autors projektam veltīja aptuveni 400 līdz 500 stundas. Šis laika ieguldījums parāda, ka pat salīdzinoši nelielas 2D spēles izveide prasa daudz darba, plānošanas un atkārtotas pārbaudes.

Projekta laikā ļoti interesanti bija veidot jaunas mehānikas, jo gandrīz katra no tām autoram bija jauna pieredze. Pirms katras sarežģītākas funkcijas izveides bija šaubas, vai to izdosies īstenot pareizi. Tas attiecās gan uz kustīgajām platformām, kontrolpunktiem un interaktīvajiem objektiem, gan uz iestatījumu saglabāšanu, skaņas sistēmu, boss cīņu un fināla titriem. Tomēr pakāpeniski problēmas tika atrisinātas, un katra jaunā mehānika palīdzēja labāk saprast Godot Engine darbības principus.

Īpašs izaicinājums bija lietotāja saskarnes izveide. Autors neuzskata dizainu par savu stiprāko pusi, tāpēc izvēlņu, pogu, logu, veikala, skapja un iestatījumu noformēšana prasīja daudz mēģinājumu. Bija svarīgi panākt, lai saskarne būtu ne tikai vizuāli pieņemama, bet arī saprotama un ērti lietojama. Rezultātā tika izveidota vienota spēles saskarne ar galveno izvēlni, līmeņu izvēlni, pauzes izvēlni, iestatījumiem, veikalu, skapi un rezultātu logu.

Par vienu no galvenajiem sasniegumiem autors uzskata to, ka izdevās izveidot ne tikai vienkāršu spēles prototipu, bet pilnvērtīgu 2D platformera tipa spēli ar vairākām savstarpēji saistītām sistēmām. Spēlē ir realizēta galvenā varoņa vadība, līmeņu izvēle, progress, zvaigžņu vērtēšana, personāžu veikals, skapis, iestatījumi, vadības maiņa, kontrolpunkti, interaktīvie objekti, īslaicīgi pastiprinājumi, ienaidnieki, kustīgās platformas, slēpto artefaktu kolekcija, boss cīņa, fināla aina un titri. Šo sistēmu savienošana vienā projektā palīdzēja autoram iegūt praktisku pieredzi spēļu izstrādē.

Lai gan autoram ir vēlme kādreiz publicēt savu spēli tādās platformās kā “Steam”, “Epic Games” vai citos digitālajos izplatīšanas kanālos, šis projekts vairāk tiek uztverts kā nozīmīga mācību pieredze. “Gaismas Meklētājs” palīdzēja apgūt Godot Engine, GDScript, 2D spēļu izstrādes principus, līmeņu veidošanu, spēles mehāniku programmēšanu un lietotāja saskarnes

izstrādi. Nākotnē autors vēlētos izmantot iegūto pieredzi jauna projekta izveidē no nulles, lai jau sākumā labāk plānotu koda struktūru, optimizāciju, ainu organizāciju un vizuālo stilu.

Turpmākajos projektos autors vēlētos vairāk uzmanības pievērst paša veidotiem grafiskajiem resursiem, lai spēles vizuālais stils pilnībā atbilstu sākotnējai idejai. Tas ļautu izvairīties no situācijām, kad spēles mehānika vai dizains jāpielāgo atrastajiem assetiem. Tāpat nākotnē būtu iespējams uzlabot animācijas, optimizēt koda struktūru, padarīt spēles sistēmas vieglāk paplašināmas un izveidot vēl kvalitatīvāku spēles pieredzi.

Kopumā kvalifikācijas darba mērķis tika sasniegts. Tika izstrādāta funkcionējoša 2D piedzīvojumu platformera spēle ar vairākām spēles mehānikām, lietotāja saskarni, progresu saglabāšanu un noslēguma plūsmu. Projekta izstrāde sniedza autoram vērtīgu praktisko pieredzi un palīdzēja labāk saprast, cik daudz dažādu elementu nepieciešams, lai izveidotu pilnvērtīgu spēli.

7. Lietoto terminu un saīsinājumu skaidrojumi

Platformer (platformeru spēle) - videospēļu žanrs, kurā spēlētājs vada varoni, kas pārvietojas pa dažādiem līmeņiem, lecot starp platformām, izvairoties no šķēršļiem un ienaidniekiem. Galvenais uzsvars tiek likts uz precīzu kustību un reakcijas ātrumu.

FP - funkcionālās prasības.

NP - nefunkcionālās prasības.

2D spēle - videospēle, kuras darbība notiek divdimensiju vidē.

Godot Engine - spēļu izstrādes dzinējs, kas tika izmantots projekta “Gaismas Meklētājs” izveidei.

GDScript - Godot Engine programmēšanas valoda, ar kuru tika veidota spēles loģika un mehānikas.

Asset - gatavs spēles resurss, piemēram, attēls, animācija, skaņa, mūzika, fons vai cits elements, kas tiek izmantots spēles izstrādē.

UI (User Interface) - lietotāja saskarne, kas ietver pogas, izvēlnes, logus, iestatījumus un citus vizuālus elementus, ar kuriem lietotājs mijiedarbojas.

Kontrollpunkts - līmeņa punkts, kas pēc aktivizēšanas kļūst par varoņa jauno atdzimšanas vietu.

Boss - īpašs pretinieks, kas parasti ir spēcīgāks par parastiem ienaidniekiem un tiek izmantots kā nozīmīgs izaicinājums spēles gaitā.

PackedScene - Godot Engine datu tips, kas ļauj saglabāt un atkārtoti ielādēt gatavas ainas vai objektu veidnes.

GameState - projekta skripts, kas pārvalda spēles progresu, iestatījumus, izvēlēto personāžu un citus saglabājamus datus.

CharacterDB - projekta datu struktūra, kurā tiek glabāta informācija par spēlē pieejamajiem personāžiem.

settings.cfg - lokāls konfigurācijas fails, kurā tiek saglabāti lietotāja iestatījumi.

progress.cfg - lokāls konfigurācijas fails, kurā tiek saglabāts spēles progress.

UML - modelēšanas valoda, kas tiek izmantota sistēmas diagrammu veidošanai.

Use Case diagramma - UML diagramma, kas parāda lietotāja iespējamās darbības sistēmā.

Activity diagramma - UML diagramma, kas parāda darbību secību sistēmā vai konkrētā procesā.

State diagramma - UML diagramma, kas parāda objekta vai sistēmas stāvokļus un pārejas starp tiem.

Klašu diagramma - UML diagramma, kas parāda sistēmas klases, to īpašības, metodes un savstarpējās saites.

Komponenšu diagramma - UML diagramma, kas parāda sistēmas galvenos komponentus un to savstarpējo sadarbību.

TST - testpiemēra identifikators testēšanas dokumentācijā.

Vector2 - datu tips, kas tiek izmantots divdimensiju pozīciju, virzienu un ātruma glabāšanai.

Artefakts - slēpts kolekcijas objekts, kuru spēlētājs var atrast līmeņu laikā un apskatīt artefaktu kolekcijas sadaļā.

Slēptais kristāls - spēles objekts, kas tiek izmantots kā paslēpts artefakts. Pēc atrašanas tas tiek saglabāts spēlētāja progresā.

Literatūras un informācijas avotu saraksts

1. **Godot Engine.** Godot Docs - oficiāla dokumentācija. Pieejams: <https://docs.godotengine.org/en/stable/>
2. **GDQuest.** Godot mācību materiāli. Pieejams: <https://www.gdquest.com/>
3. **Youtube.** Godot Engine, GDScript un spēļu izstrādes mācību video materiāli. Pieejams: <https://www.youtube.com/>
4. **GDScript.** GDScript mācību materiāli. Pieejams: <https://gdscript.com/>
5. **itch.io.** Spēļu resursu platforma. Pieejams: <https://itch.io/>
6. **Kenney.** Spēļu resursu platforma. Pieejams: <https://kenney.nl/>
7. **OpenGameArt.** Spēļu resursu platforma. Pieejams: <https://opengameart.org/>
8. **Pixabay.** Skaņu efektu un mūzikas platforma. Pieejams: <https://pixabay.com/>
9. **Mixkit.** Skaņu efektu un mūzikas platforma. Pieejams: <https://mixkit.co/>
10. **Godot Forum.** Godot kopienas forums. Pieejams: <https://forum.godotengine.org/>

Pielikumi

1. Pielikums

Super lēciena pastiprinājuma pirmais koda fragments

```
1  # Skripts paplašina Area2D mezglu, jo pastiprinājums darbojas kā zona,
2  # kurā tiek pārbaudīts, vai spēlētājs tai ir pietuvojies.
3  extends Area2D
4
5  # Signāls tiek izsaukts brīdī, kad spēlētājs savāc šo pastiprinājumu.
6  signal picked_up
7
8  # Ikona, kas vizuāli attēlo pastiprinājuma objektu līmenī.
9  @onready var icon: Sprite2D = $Icon
10
11 # Taimeris, kas nosaka, pēc cik laika pastiprinājums atkal parādīsies līmenī.
12 @onready var respawn_timer: Timer = $RespawnTimer
13
14 # Atsauce uz mijiedarbības norādi, kas tiek parādīta spēlētājam,
15 # kad viņš atrodas pietiekami tuvu objektam.
16 @onready var interaction_hint: CanvasLayer = get_parent().get_node_or_null("InteractionHint")
17
18 # Atsauce uz spēlētāja objektu, lai varētu piešķirt pastiprinājumu.
19 @onready var player: CharacterBody2D = get_parent().get_node_or_null("Player")
20
21 # skaņa, kas tiek atskaņota, kad spēlētājs savāc pastiprinājumu.
22 @onready var pickup_sound: AudioStreamPlayer = $PickUpSound
23
24 # Mainīgais nosaka, vai spēlētājs pašlaik atrodas objekta darbības zonā.
25 var player_in_range := false
26
27 # Mainīgais nosaka, vai pastiprinājums jau ir aktivizēts.
28 var activated := false
29
30 # Pastiprinājuma darbības ilgums sekundēs.
31 @export var super_jump_duration: float = 5.0
32
33 # Ja vērtība ir true, objekts darbojas tikai vienu reizi un pēc savākšanas tiek izdzēsts.
34 @export var one_shot := false
35
36
37 func _ready() -> void:
38     # Savieno ieiešanas zonā signālu ar funkciju,
39     # kas nosaka, kad spēlētājs tuvojas objektam.
40     body_entered.connect(_on_body_entered)
41
42     # Savieno iziešanas no zonas signālu ar funkciju,
43     # kas noslēpj mijiedarbības norādi.
44     body_exited.connect(_on_body_exited)
45
46     # Savieno taimera signālu ar funkciju,
47     # kas atjauno objektu pēc noteikta laika.
48     respawn_timer.timeout.connect(_on_respawn_timer_timeout)
49
50     # Ja spēlētājam ir "died" signāls, tas tiek savienots ar funkciju,
51     # lai spēlētāja nāves gadījumā atiestatītu vai izdzēstu pastiprinājumu.
52     if player != null and player.has_signal("died") and not player.died.is_connected(_on_player_died):
53         player.died.connect(_on_player_died)
54
55     # Sākumā mijiedarbības norāde netiek rādīta.
56     if interaction_hint != null:
57         interaction_hint.visible = false
58
```

Super lēciena pastiprinājuma otrais koda fragments

```

60  ▾ func _process(_delta: float) -> void:
61    ▸ | # Ja pastiprinājums jau ir aktivizēts, tālākā apstrāde nav vajadzīga.
62    ▾ ▸ | if activated:
63      ▸ | ▸ | return
64
65    ▾ ▸ | # Ja spēlētājs atrodas darbības zonā un nospiež mijiedarbības taustiņu,
66      ▸ | # tiek aktivizēts pastiprinājums.
67    ▾ ▸ | if player_in_range and Input.is_action_just_pressed("interact"):
68      ▸ | ▸ | activated = true
69
70    ▾ ▸ | ▸ | # Ja spēlētāja objektam ir metode apply_super_jump(),
71      ▸ | ▸ | # tiek piešķirts super jump efekts uz noteiktu laiku.
72    ▾ ▸ | ▸ | if player != null and player.has_method("apply_super_jump"):
73      ▸ | ▸ | ▸ | player.apply_super_jump(super_jump_duration)
74
75    ▾ ▸ | ▸ | # Spēlētāja kustība tiek apstādināta,
76      ▸ | ▸ | # lai savākšanas brīdis būtu korektāks.
77    ▾ ▸ | ▸ | if player != null:
78      ▸ | ▸ | ▸ | player.velocity = Vector2.ZERO
79
80      ▸ | ▸ | # Pēc savākšanas mijiedarbības norāde tiek paslēpta.
81    ▾ ▸ | ▸ | if interaction_hint != null:
82      ▸ | ▸ | ▸ | interaction_hint.visible = false
83
84      ▸ | ▸ | # Atskaņo savākšanas skaņu.
85    ▾ ▸ | ▸ | if pickup_sound != null:
86      ▸ | ▸ | ▸ | pickup_sound.play()
87
88      ▸ | ▸ | # Ikona tiek paslēpta, jo objekts pašlaik ir savākts.
89      ▸ | ▸ | icon.visible = false
90
91      ▸ | ▸ | # Tiek izslēgta uzraudzība, lai objektu nevarētu savākt atkārtoti uzreiz.
92      ▸ | ▸ | monitoring = false
93
94      ▸ | ▸ | # Tiek izsaukts signāls, ka objekts ir savākts.
95      ▸ | ▸ | picked_up.emit()
96
97    ▾ ▸ | ▸ | # Ja šis ir vienreiz lietojams objekts,
98      ▸ | ▸ | # tas pēc skaņas atskaņošanas tiek izdzēsts.
99    ▾ ▸ | ▸ | if one_shot:
100     ▸ | ▸ | ▸ | set_process(false)
101
102    ▾ ▸ | ▸ | ▸ | if pickup_sound != null and pickup_sound.stream != null and pickup_sound.playing:
103     ▸ | ▸ | ▸ | ▸ | await pickup_sound.finished
104
105     ▸ | ▸ | ▸ | queue_free()
106    ▾ ▸ | ▸ | else:
107    ▾ ▸ | ▸ | ▸ | # Ja objekts nav vienreiz lietojams,
108     ▸ | ▸ | ▸ | # tiek palaists taimeris tā atkārtotai parādīšanai.
109     ▸ | ▸ | ▸ | respawn_timer.start()
110

```

Super lēciena pastiprinājuma trešais koda fragments

```
112  ▾ func _on_body_entered(body: Node) -> void:
113    ▸ # Ja monitoring ir izslēgts, objekts nereaģē uz ieiešanu zonā.
114  ▾ ▸ if not monitoring:
115    ▸ ▸ return
116
117  ▾ ▸ # Ja zonā ieiet spēlētājs, tiek atzīmēts,
118    ▸ # ka viņš atrodas pietiekami tuvu mijiedarbībai.
119  ▾ ▸ if body.name == "Player":
120    ▸ ▸ player_in_range = true
121
122    ▸ ▸ # Parāda mijiedarbības norādi.
123  ▾ ▸ ▸ if interaction_hint != null:
124    ▸ ▸ ▸ interaction_hint.visible = true
125
126
127  ▾ func _on_body_exited(body: Node) -> void:
128  ▾ ▸ # Ja spēlētājs iziet no objekta darbības zonas,
129    ▸ # mijiedarbība vairs nav iespējama.
130  ▾ ▸ if body.name == "Player":
131    ▸ ▸ player_in_range = false
132
133    ▸ ▸ # Paslēpj mijiedarbības norādi.
134  ▾ ▸ ▸ if interaction_hint != null:
135    ▸ ▸ ▸ interaction_hint.visible = false
136
137
138  ▾ func _on_respawn_timer_timeout() -> void:
139    ▸ # Pēc taimera beigām objekts tiek atjaunots sākotnējā stāvoklī.
140    ▸ activated = false
141    ▸ player_in_range = false
142    ▸ icon.visible = true
143    ▸ monitoring = true
```

Super lēciena pastiprinājuma ceturtais koda fragments

```
146  ▾ func force_reset() -> void:
147  ▾ >| # Funkcija piespiedu kārtā atiestata pastiprinājumu,
148    >| # piemēram, spēlētāja nāves gadījumā.
149    >| activated = false
150    >| player_in_range = false
151
152    >| # Ja taimeris darbojas, tas tiek apturēts.
153  ▾ >| if respawn_timer != null:
154    >| >| respawn_timer.stop()
155
156    >| # Atjauno objekta redzamību un darbību.
157    >| icon.visible = true
158    >| monitoring = true
159
160    >| # Paslēpj mijiedarbības norādi.
161  ▾ >| if interaction_hint != null:
162    >| >| interaction_hint.visible = false
163
164
165  ▾ func _on_player_died() -> void:
166  ▾ >| # Ja objekts ir vienreiz lietojams,
167    >| # spēlētāja nāves gadījumā tas tiek pilnībā izdzēsts.
168  ▾ >| if one_shot:
169  ▾ >| >| if interaction_hint != null:
170    >| >| >| interaction_hint.visible = false
171
172    >| >| visible = false
173    >| >| monitoring = false
174    >| >| set_process(false)
175    >| >| queue_free()
176    >| >| return
177
178    >| # Pretējā gadījumā objekts tiek vienkārši atiestatīts.
179    >| force_reset()
```


Kristāla pirmais koda fragments

```

1  # Skripts paplašina Area2D mezglu, jo kristāls darbojas kā mijiedarbības zona.
2  extends Area2D
3
4  # Signāls tiek izsaukts pēc kristāla iznīcināšanas animācijas beigām.
5  # Līmeņa skripts šo signālu izmanto, lai sāktu līmeņa noslēguma plūsmu.
6  signal collected
7
8  # Kristāla animētais sprite. Tas atskaņo idle un destruction animācijas.
9  @onready var sprite: AnimatedSprite2D = $AnimatedSprite2D
10
11 # Kopīgā mijiedarbības norāde, kas tiek parādīta, kad spēlētājs var aktivizēt kristālu.
12 @onready var interaction_hint: CanvasLayer = get_parent().get_node_or_null("InteractionHint")
13
14 # Skana, kas tiek atskaņota kristāla aktivizēšanas brīdī.
15 @onready var destruction_sound: AudioStreamPlayer = $CrystalDestructionSound
16
17 # Atsauce uz spēlētāju, lai varētu pārbaudīt viņa stāvokli un bloķēt vadību cutscene laikā.
18 @onready var player: CharacterBody2D = get_parent().get_node_or_null("Player")
19
20 # Nosaka, vai spēlētājs pašlaik atrodas kristāla darbības zonā.
21 var player_in_range := false
22
23 # Nosaka, vai kristāls jau ir aktivizēts.
24 # Tas neļauj vienu un to pašu kristālu aktivizēt vairākas reizes.
25 var activated := false
26
27 # Ja locked ir true, kristālu nevar aktivizēt.
28 # Finālajā līmenī tas tiek izmantots, lai bloķētu kristālu līdz boss uzvarēšanai.
29 @export var locked := false
30
31
32 # func _ready() -> void:
33     # Sākumā kristāls atskaņo idle animāciju.
34     sprite.play("idle")
35
36     # Signāli nosaka, kad spēlētājs ieiet kristāla zonā vai iziet no tās.
37     body_entered.connect(_on_body_entered)
38     body_exited.connect(_on_body_exited)
39
40     # Signāls tiek izmantots, lai pēc destruction animācijas beigām paziņotu līmenim,
41     # ka kristāls ir savākts.
42     sprite.animation_finished.connect(_on_animation_finished)
43
44     # Sākumā mijiedarbības norāde netiek rādīta.
45     if interaction_hint != null:
46         interaction_hint.visible = false
47

```

Kristāla otrais koda fragments

```

49  func _process(_delta: float) -> void:
50  >| # Ja kristāls ir bloķēts, spēlētājs to nevar aktivizēt.
51  >| # Norāde tiek paslēpta, lai lietotājs neredzētu mijiedarbības iespēju.
52  >| if locked:
53  >| >| if interaction_hint != null and player_in_range:
54  >| >| >| interaction_hint.visible = false
55  >| >| return
56
57  >| # Ja kristāls jau ir aktivizēts, atkārtota mijiedarbība nav iespējama.
58  >| if activated:
59  >| >| if interaction_hint != null and player_in_range:
60  >| >| >| interaction_hint.visible = false
61  >| >| return
62
63  >| # Ja spēlētājs atrodas kristāla zonā, tiek atjaunināta mijiedarbības norāde.
64  >| if player_in_range:
65  >| >| _update_interaction_hint()
66
67  >| >| # Kristāls tiek aktivizēts tikai tad, ja spēlētājs var mijiedarboties
68  >| >| # un nospiež mijiedarbības taustiņu.
69  >| >| if _can_interact() and Input.is_action_just_pressed("interact"):
70  >| >| >| _activate_crystal()
71
72
73  func _can_interact() -> bool:
74  >| # Bloķētu kristālu nevar aktivizēt.
75  >| if locked:
76  >| >| return false
77
78  >| # Spēlētājam jāatrodas kristāla zonā.
79  >| if not player_in_range:
80  >| >| return false
81
82  >| # Ja spēlētāja objekts nav atrasts, mijiedarbība nav iespējama.
83  >| if player == null:
84  >| >| return false
85
86  >| # Kristālu var aktivizēt tikai tad, ja spēlētājs stāv uz zemes.
87  >| # Tas novērš situāciju, kad līmenis tiek pabeigts lēciena laikā.
88  >| if not player.is_on_floor():
89  >| >| return false
90
91  >| return true
92
93
94  func _update_interaction_hint() -> void:
95  >| # Ja norādes mezgls nav atrasts, funkcija neturpina darbu.
96  >| if interaction_hint == null:
97  >| >| return
98
99  >| # Norāde ir redzama tikai tad, ja spēlētājs tiešām var aktivizēt kristālu.
100 >| interaction_hint.visible = _can_interact()
101

```

Kristāla trešais koda fragments

```

102
103 < func _activate_crystal() -> void:
104     > # Atzīmē, ka kristāls ir aktivizēts.
105     > activated = true
106
107     > # Spēlētāja vadība tiek bloķēta, lai sāktos cutscene vai līmeņa beigu plūsma.
108 < > if player != null:
109     > > player.cutscene_lock = true
110     > > player.velocity = Vector2.ZERO
111
112     > # Pēc aktivizēšanas norāde tiek paslēpta.
113 < > if interaction_hint != null:
114     > > interaction_hint.visible = false
115
116     > # Atskaņo kristāla iznīcināšanas skaņu.
117 < > if destruction_sound != null:
118     > > destruction_sound.play()
119
120     > # Palaiž kristāla iznīcināšanas animāciju.
121     > sprite.play("destruction")
122
123
124 < func _on_body_entered(body: Node) -> void:
125     > # Ja spēlētājs ieiet kristāla zonā, tiek atļauta mijiedarbības pārbaude.
126 < > if body.name == "Player":
127     > > player_in_range = true
128
129     > > # Ja spēlētāja atsauce vēl nav saglabāta, tā tiek iegūta no ienākušā objekta.
130 < > > if player == null:
131     > > > player = body as CharacterBody2D
132
133     > > # Uzreiz atjaunina norādi atkarībā no tā, vai kristālu var aktivizēt.
134     > > _update_interaction_hint()
135
136
137 < func _on_body_exited(body: Node) -> void:
138     > # Ja spēlētājs iziet no zonas, mijiedarbība vairs nav iespējama.
139 < > if body.name == "Player":
140     > > player_in_range = false
141
142     > > # Norāde tiek paslēpta.
143 < > > if interaction_hint != null:
144     > > > interaction_hint.visible = false
145
146
147 < func _on_animation_finished() -> void:
148 < > # Kad destruction animācija beidzas, tiek izsaukts collected signāls.
149     > # Pēc tam līmeņa skripts var parādīt rezultātu logu vai sākt fināla titrus.
150 < > if sprite.animation == "destruction":
151     > > collected.emit()
152     > > hide()
153
154
155 < func set_locked(value: bool) -> void:
156     > # Funkcija ļauj līmeņa skriptam bloķēt vai atbloķēt kristālu.
157     > locked = value
158
159     > # Ja kristāls tiek bloķēts, norāde tiek paslēpta.
160 < > if locked and interaction_hint != null and player_in_range:
161     > > interaction_hint.visible = false

```

Lūztošās platformas pirmais koda fragments

```

1  ▾ # Skripts paplašina StaticBody2D mezglu, jo platforma parasti ir ciets objekts,
2    # uz kura spēlētājs var stāvēt, līdz tā salūzt.
3    extends StaticBody2D
4
5    # Aizture sekundēs pirms platformas kolīzijas izslēgšanas.
6    @export var break_delay := 0.45
7
8    # Laiks sekundēs, pēc kura platforma atkal parādās.
9    @export var respawn_time := 2.0
10
11   # Platformas animētais sprite. Tas atskaņo idle un break animācijas.
12   @onready var sprite: AnimatedSprite2D = $AnimatedSprite2D
13
14   # Galvenā kolīzija, kas ļauj spēlētājam stāvēt uz platformas.
15   @onready var solid_collision: CollisionShape2D = $CollisionShape2D
16
17   # Papildu zona, kas nosaka, kad spēlētājs ir uzkāpis uz platformas.
18   @onready var trigger_area: Area2D = $TriggerArea
19
20   # Trigger zonas kolīzija.
21   @onready var trigger_collision: CollisionShape2D = $TriggerArea/CollisionShape2D
22
23   # Taimeris, kas tiek izmantots platformas atjaunošanai pēc salūšanas.
24   @onready var respawn_timer: Timer = $RespawnTimer
25
26   # Nosaka, vai platforma pašlaik atrodas salūšanas procesā.
27   var breaking := false
28
29   # Nosaka, vai platforma jau ir salūzusī.
30   var broken := false
31
32
33   ▾ func _ready() -> void:
34     >| # Sākumā platforma atskaņo idle animāciju.
35     >| sprite.play("idle")
36
37   ▾ >| # Savieno trigger zonas signālu ar funkciju,
38     >| # kas reaģē, kad uz platformas uzkāpj spēlētājs.
39     >| trigger_area.body_entered.connect(_on_trigger_body_entered)
40
41   ▾ >| # Savieno taimera signālu ar funkciju,
42     >| # kas atjauno platformu pēc noteikta laika.
43     >| respawn_timer.timeout.connect(_on_respawn_timer_timeout)
44
45     >| # Taimeris tiek izmantots kā vienreizējs taimeris.
46     >| respawn_timer.one_shot = true
47
48
49   ▾ func _on_trigger_body_entered(body: Node) -> void:
50   ▾ >| # Ja platforma jau salūzt vai ir salūzusī,
51     >| # tā atkārtoti nereaģē uz ieiešanu zonā.
52   ▾ >| if breaking or broken:
53     >| >| return
54
55     >| # Platforma reaģē tikai uz spēlētāju.
56   ▾ >| if body.name != "Player":
57     >| >| return
58
59     >| # Ja uz platformas uzkāpj spēlētājs, sākas salūšanas process.
60     >| _start_breaking()
61

```


Lūztošās platformas otrais koda fragments

```

63  ▾ func _start_breaking() -> void:
64    >| # Atzīmē, ka platforma pašlaik salūzt.
65    >| breaking = true
66
67  ▾ >| # Trigger zona tiek izslēgta,
68    >| # lai salūšanas process netiktu palaists atkārtoti.
69    >| trigger_area.set_deferred("monitoring", false)
70
71    >| # Tiek palaista platformas salūšanas animācija.
72    >| sprite.play("break")
73
74  ▾ >| # Pirms kolīzijas izslēgšanas tiek nogaidīta neliela aizture,
75    >| # lai spēlētājs redzētu salūšanas efektu.
76    >| await get_tree().create_timer(break_delay).timeout
77
78  ▾ >| # Pēc aiztures platforma vairs nav cieta,
79    >| # un spēlētājs tai var izkrist cauri.
80    >| solid_collision.set_deferred("disabled", true)
81
82    >| # Sagaida, līdz salūšanas animācija pilnībā beidzas.
83    >| await sprite.animation_finished
84
85    >| # Platforma tiek paslēpta.
86    >| visible = false
87
88    >| # Atzīmē, ka platforma ir salūzusī.
89    >| broken = true
90
91    >| # Salūšanas process ir beidzies.
92    >| breaking = false
93
94    >| # Tiek palaists taimeris platformas atkārtotai parādīšanai.
95    >| respawn_timer.start(respawn_time)
96
97
98  ▾ func _on_respawn_timer_timeout() -> void:
99    >| # Pēc taimera beigām platforma atkal kļūst redzama.
100   >| visible = true
101
102  ▾ >| # Atkal ieslēdz galveno kolīziju,
103    >| # lai spēlētājs varētu stāvēt uz platformas.
104    >| solid_collision.set_deferred("disabled", false)
105
106    >| # Atkal ieslēdz trigger zonas uzraudzību.
107    >| trigger_area.set_deferred("monitoring", true)
108
109    >| # Atiestata platformas stāvokļus.
110    >| broken = false
111    >| breaking = false
112
113    >| # Platforma atgriežas sākotnējā idle animācijā.
114    >| sprite.play("idle")

```